

Hardware and Software Fundamentals of Intelligent Computing Systems

AI System Fundamentals & Heterogeneous Computing

Guojie Luo

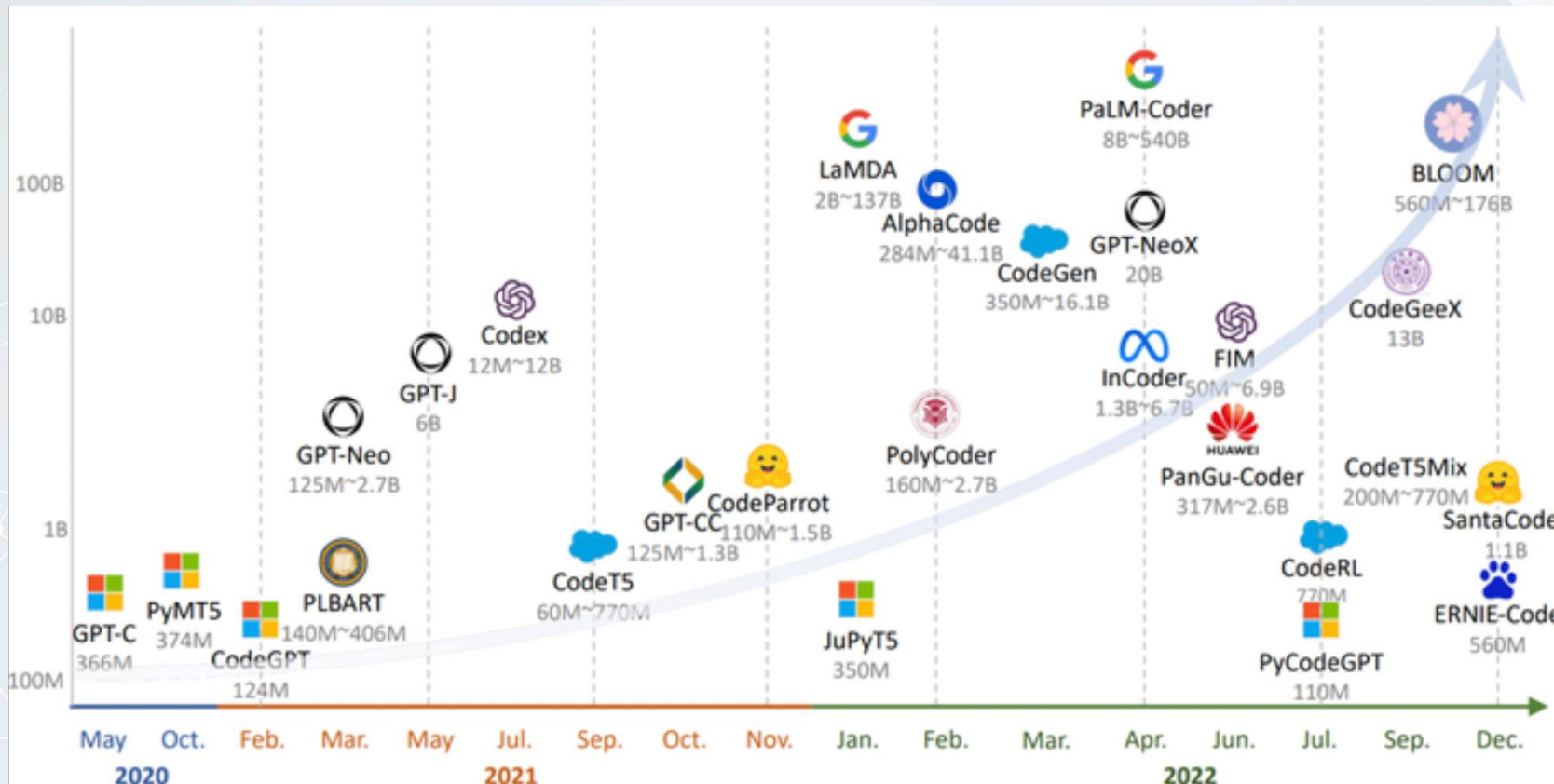
2026/8/7

- Communication Primitives
- Communication Topology Optimization

Scaling Law of LLMs



- Model performance improves predictably as a power-law function of compute, data size, and parameter count.

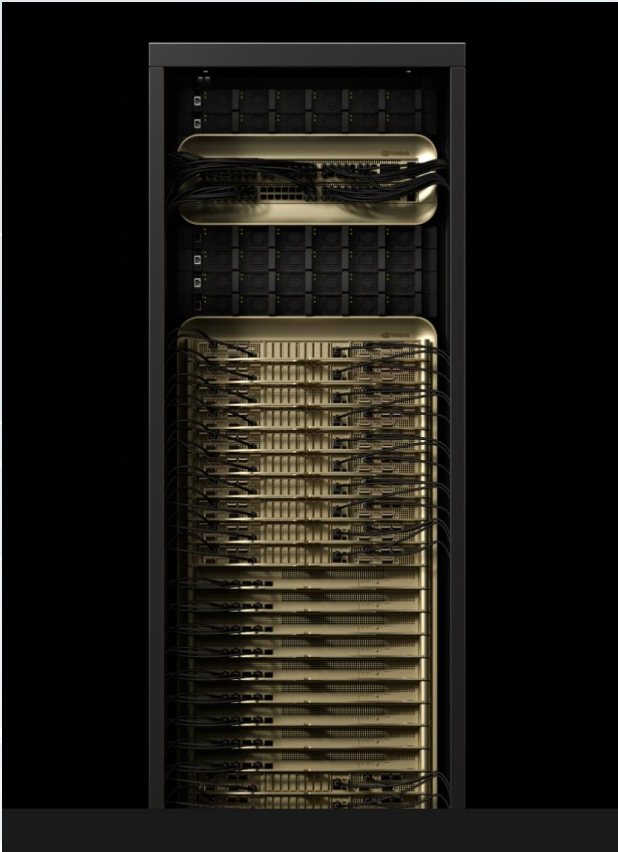


[1] <https://arxiv.org/abs/2001.08361>

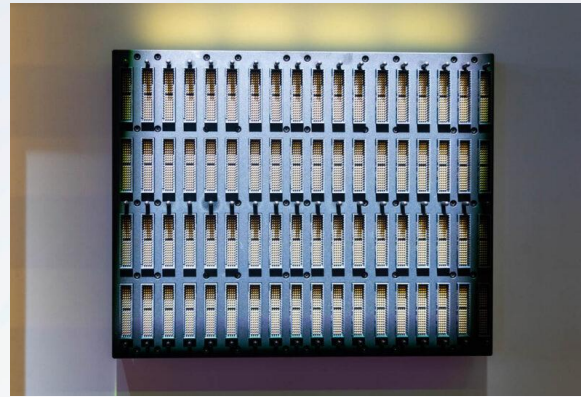
[2] <https://dnacap.fund/insights/exploring-the-landscape-of-large-language-models>

AI computing systems are also scaling

- Scale-up supernodes are on the rise...



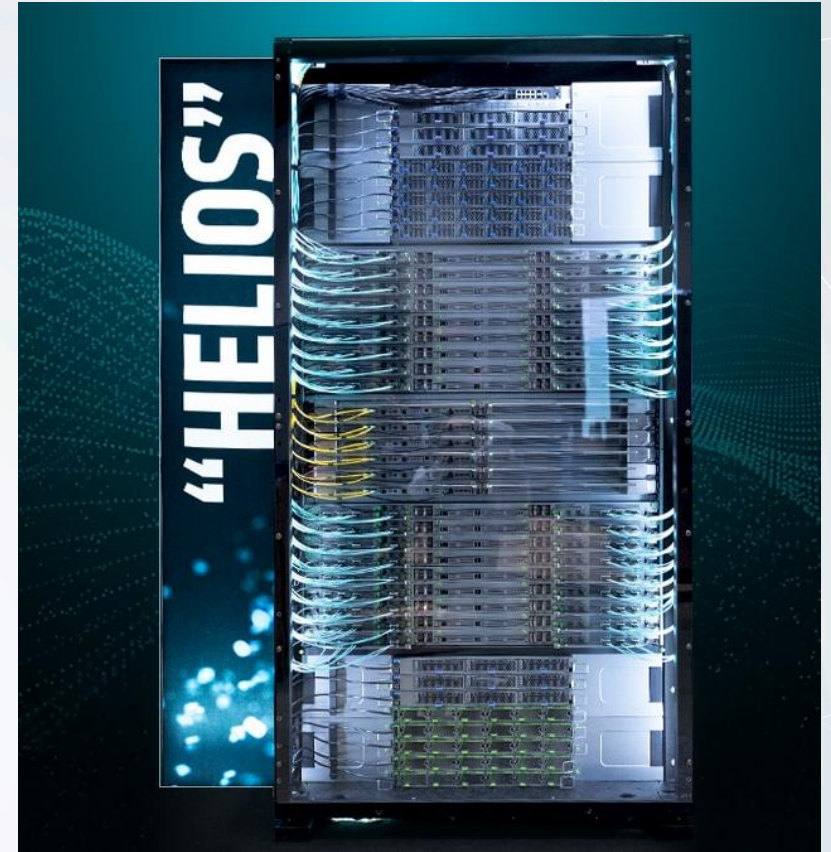
NVL72



NVL576



Huawei CloudMatrix384



AMD Helios

Mapping LLMs on multiple GPUs/NPUs...



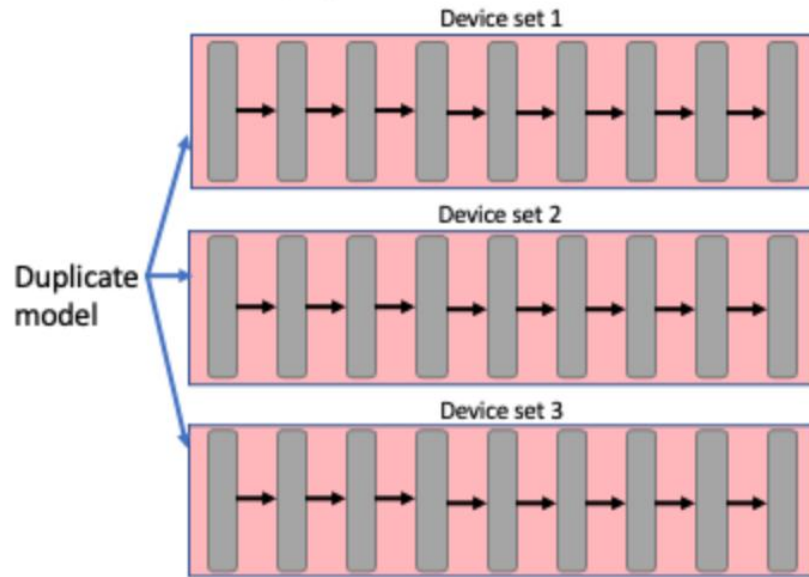
BAII
智源研究院



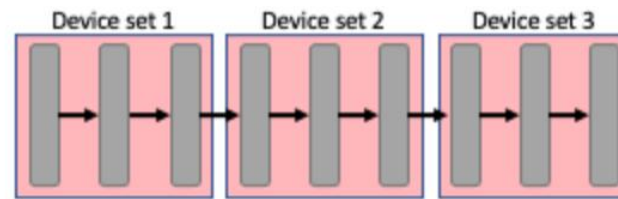
北京大学
PEKING UNIVERSITY

- Multi-dimension Parallelisms

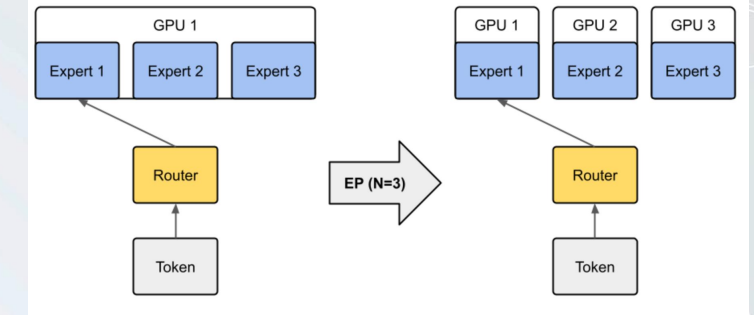
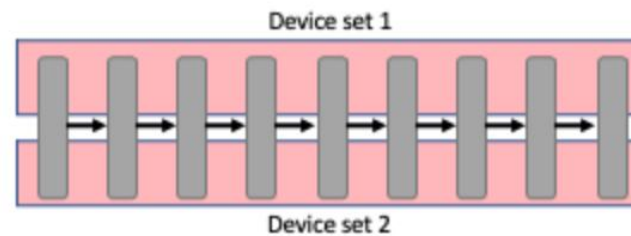
Data Parallelism (DP)



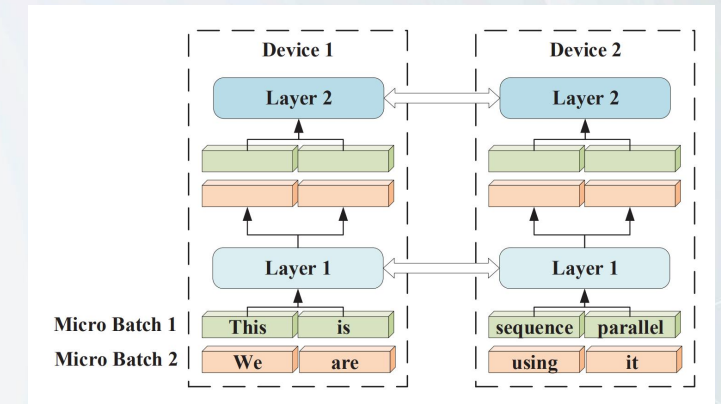
Pipeline Parallelism (PP)



Tensor Parallelism (TP)



Expert Parallelism

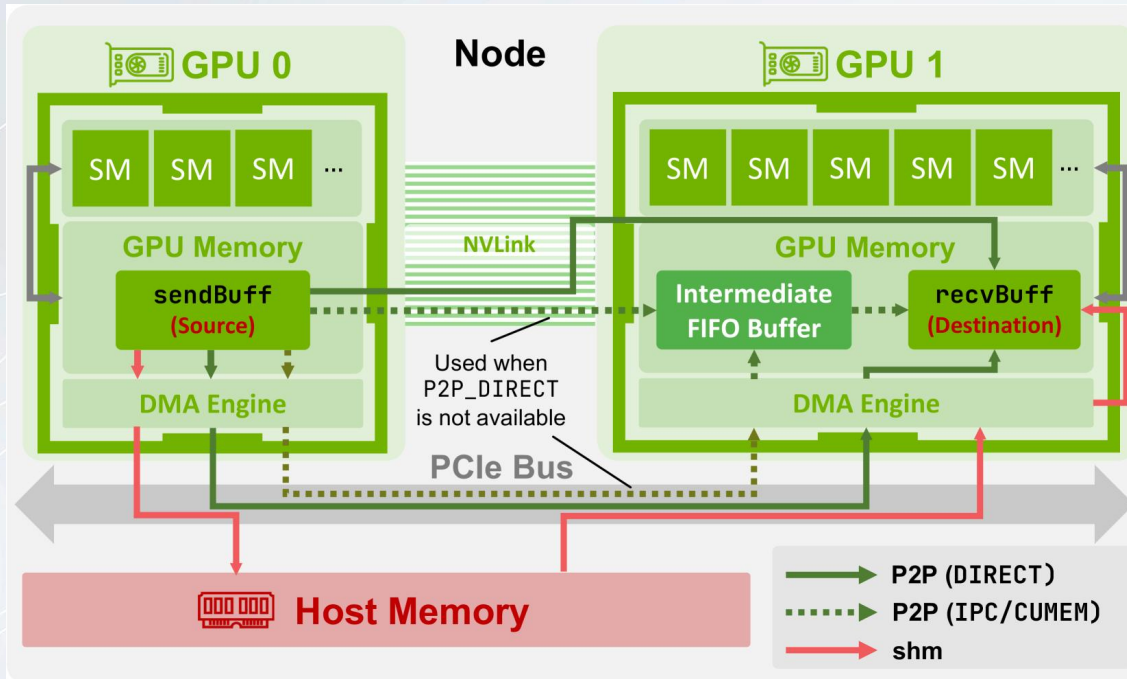


Sequence Parallelism

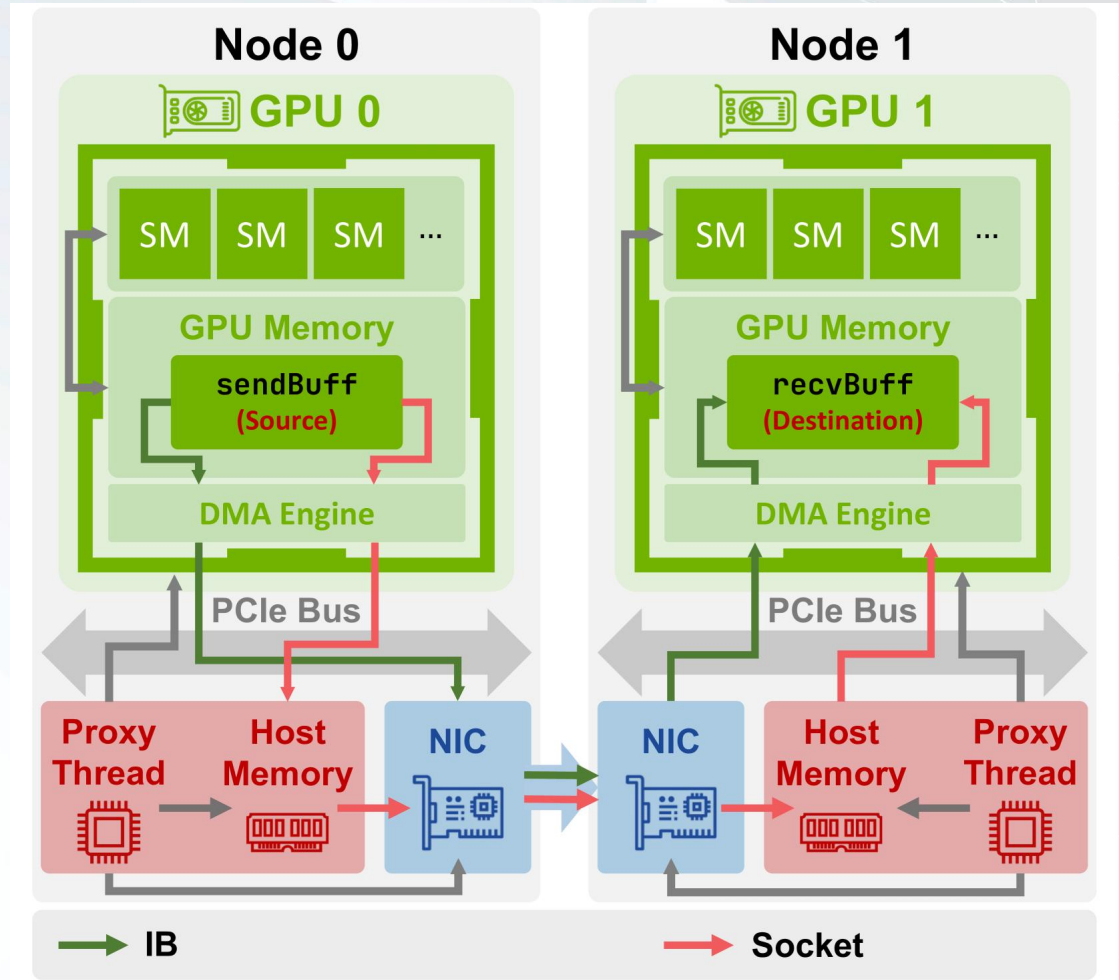
- [1] <https://robotchinwag.com/posts/demystifying-tensor-parallelism/>
- [2] <https://docs.nvidia.com/nemo/megatron-bridge/0.2.0/parallelisms.html>
- [3] <https://aclanthology.org/2023.acl-long.134.pdf>

Communication datapath in the AI systems

- Scale-up: NVLINK, HCCS, UnifiedBus
- Scale-out: InfiniBand (IB), RoCE



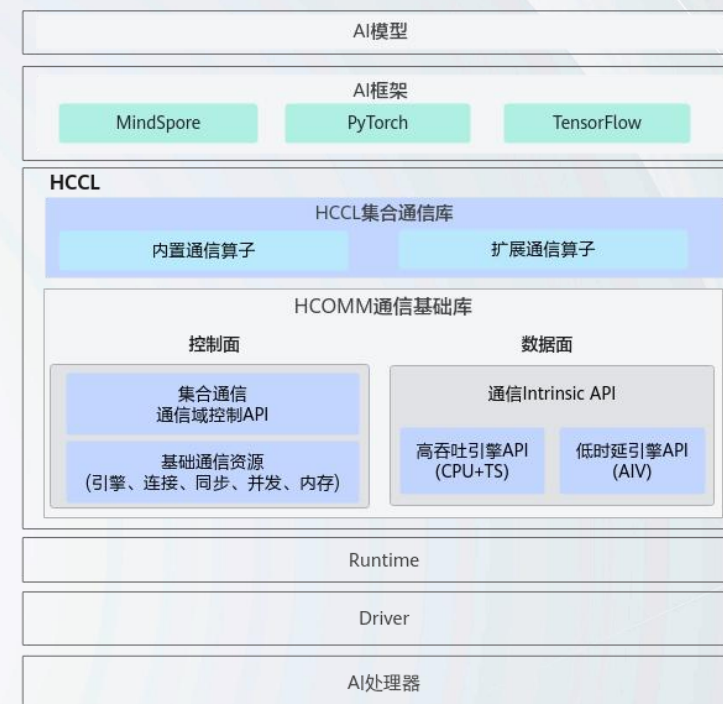
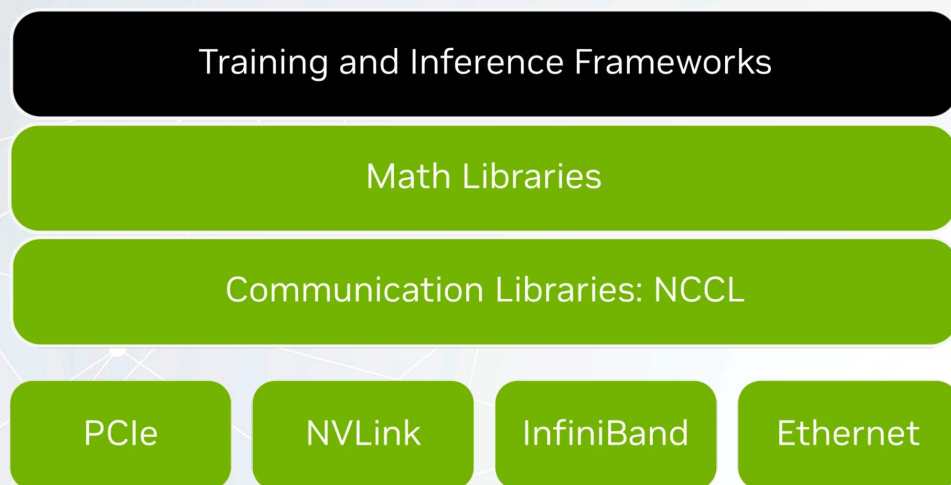
Intra-node GPU communications via NVLINK or PCIe



Inter-node communications via RDMA NIC (IB)

Collective v.s. Point-to-point communications

- Collective operations involve communication among a group of processes rather than just between pairs of processes. e.g., AllReduce, AllGather, Bcast
- Point-to-point: Send/Recv (Two-side), Put/Get (One-side)

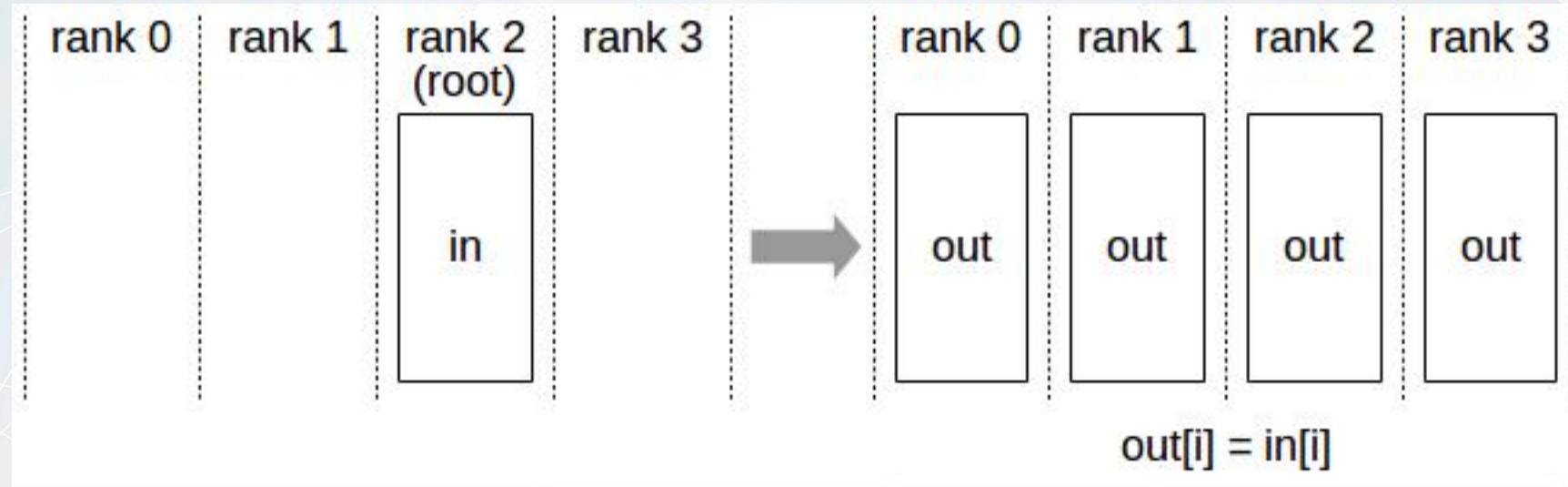


[1] <https://developer.nvidia.com/blog/scaling-ai-inference-performance-and-flexibility-with-nvidia-nvlink-and-nvlink-fusion/>

[2] <https://www.hiascend.com/developer/techArticles/20240809-1>

Broadcast

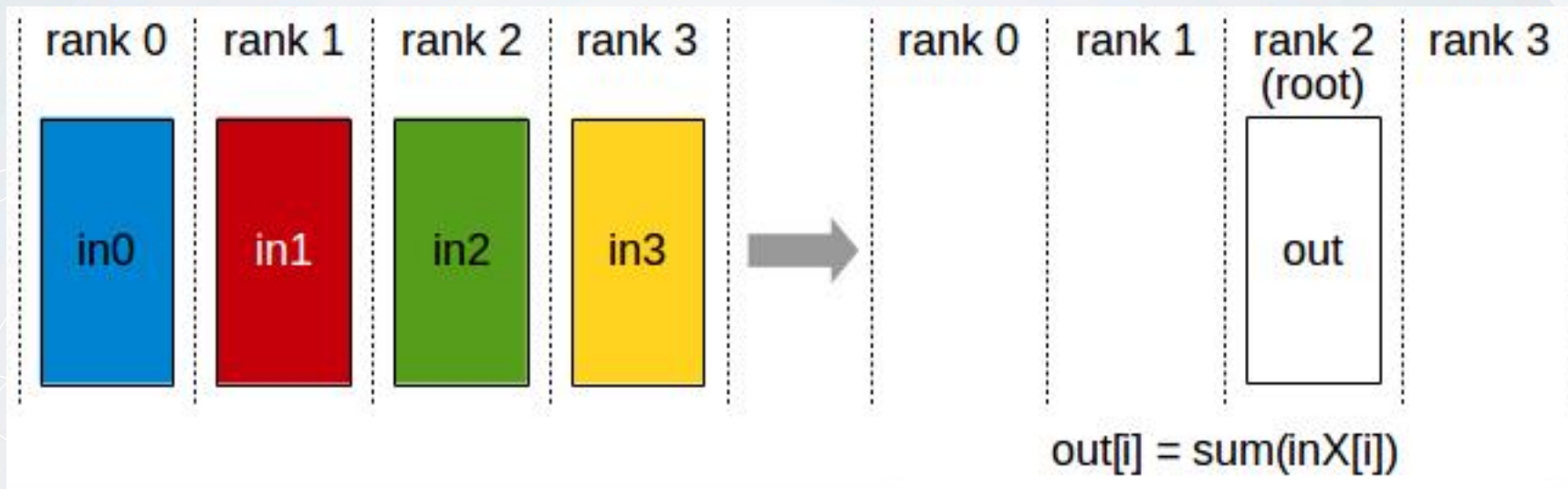
- Operations: copy an N-element buffer from the root rank (2) to all the ranks (0-3).



Reduce



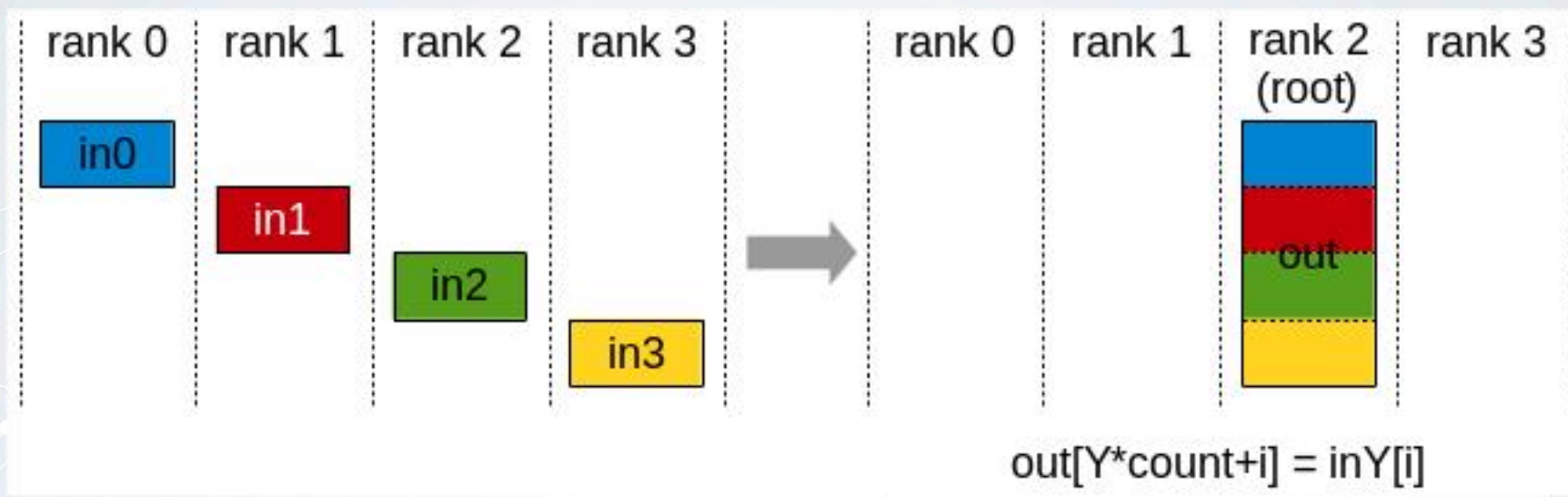
- Operations: performs reduction on data (for example, sum, min, max) across devices and stores the result in the receive buffer of a specified root rank.



Gather

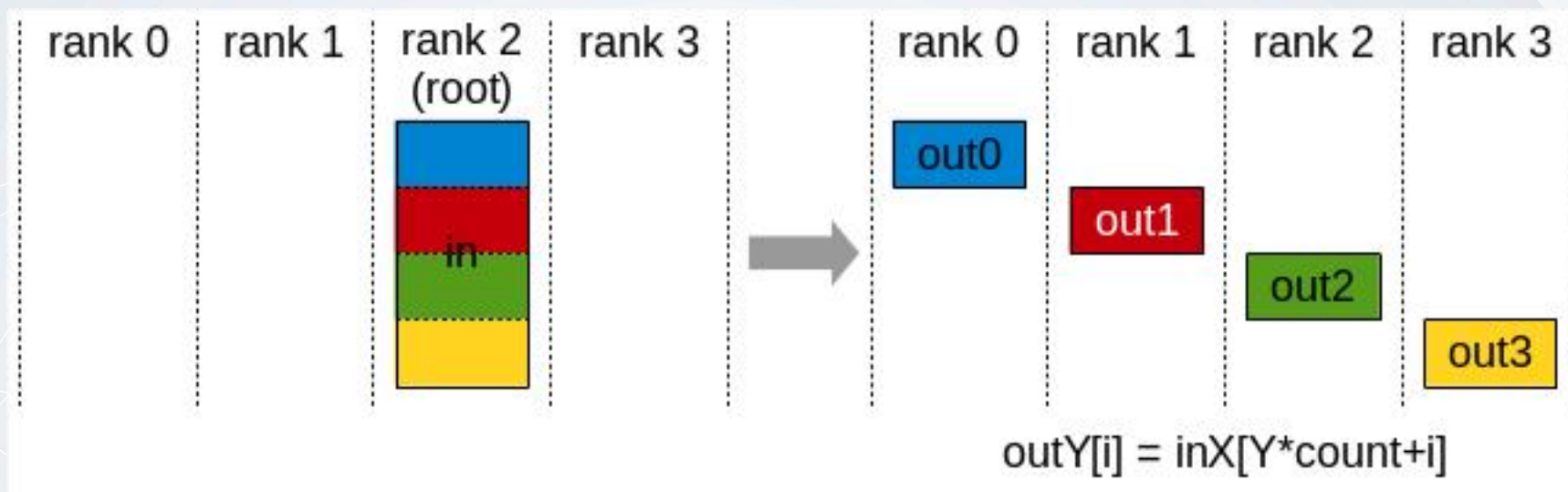


- Operations: gathers N values from k ranks into an output buffer on the root rank of size $k \cdot N$.



Scatter

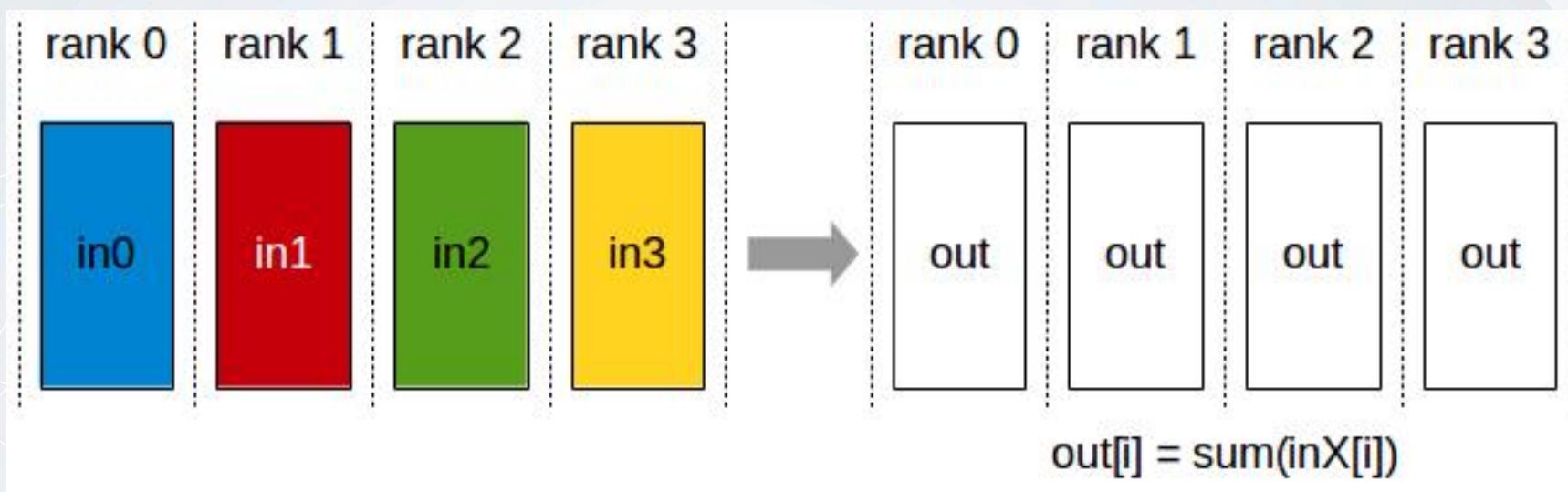
- Operations: distributes a total of $N*k$ values from the root rank to k ranks, each rank receiving N values.



AllReduce



- Operations: performs reductions on data (for example, sum, min, max) across devices and stores the result in the receive buffer of every rank.



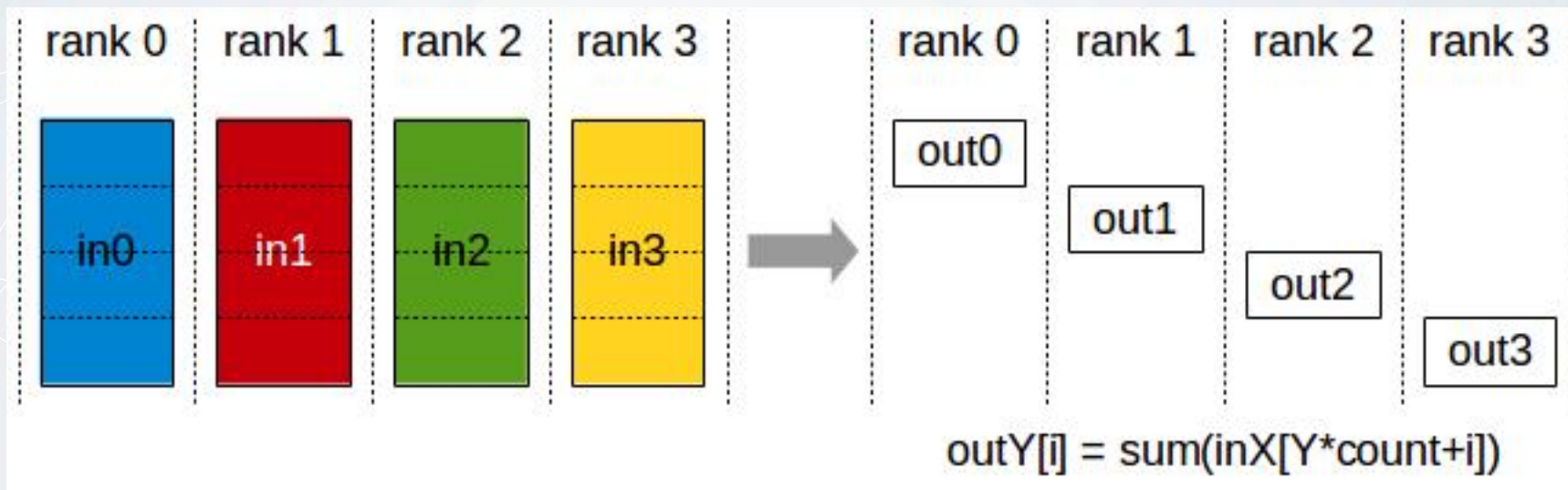
AllGather

- Operations: gathers N values from k ranks into an output buffer of size $k*N$, and distributes that result to all ranks.
 - The output is ordered by the rank index.
 - This operation is therefore impacted by a different rank to device mapping.



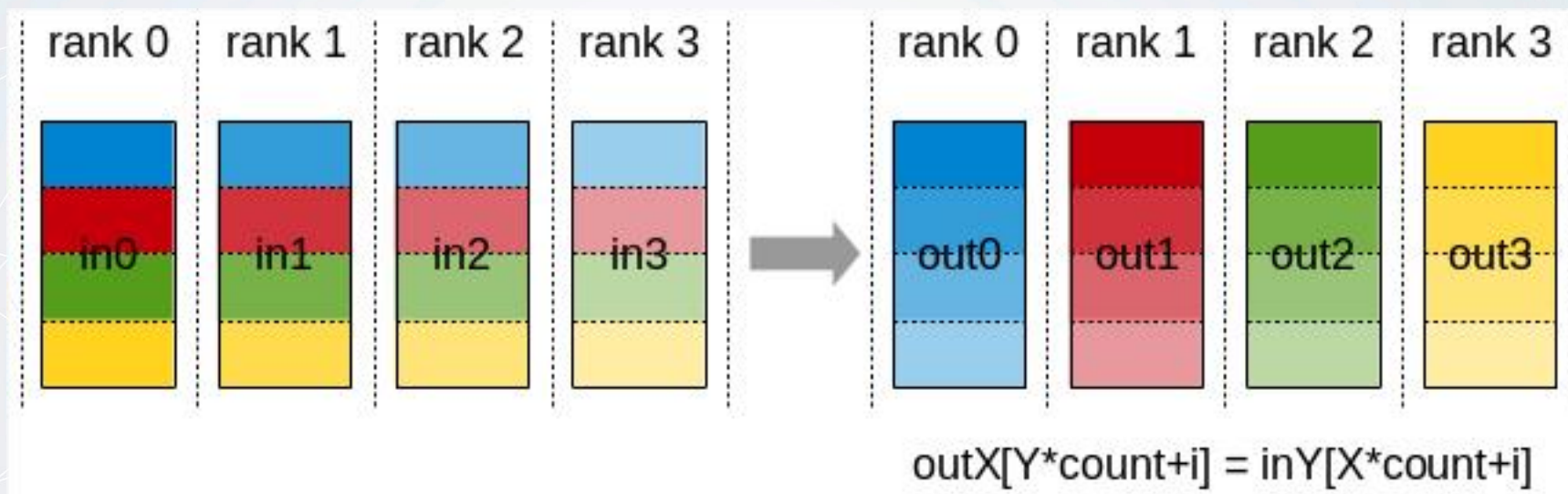
ReduceScatter

- Operations: performs the same operation as Reduce, except that the result is scattered in equal-sized blocks between ranks, each rank getting a chunk of data based on its rank index.
 - This operation is therefore impacted by a different rank to device mapping.



AlltoAll

- Operations: each rank provides an input buffer of size N values, where the j -th chunk of N/k values is sent to destination rank j . Each rank receives an output buffer of size N values, where the i -th chunk of N/k values comes from source rank i .

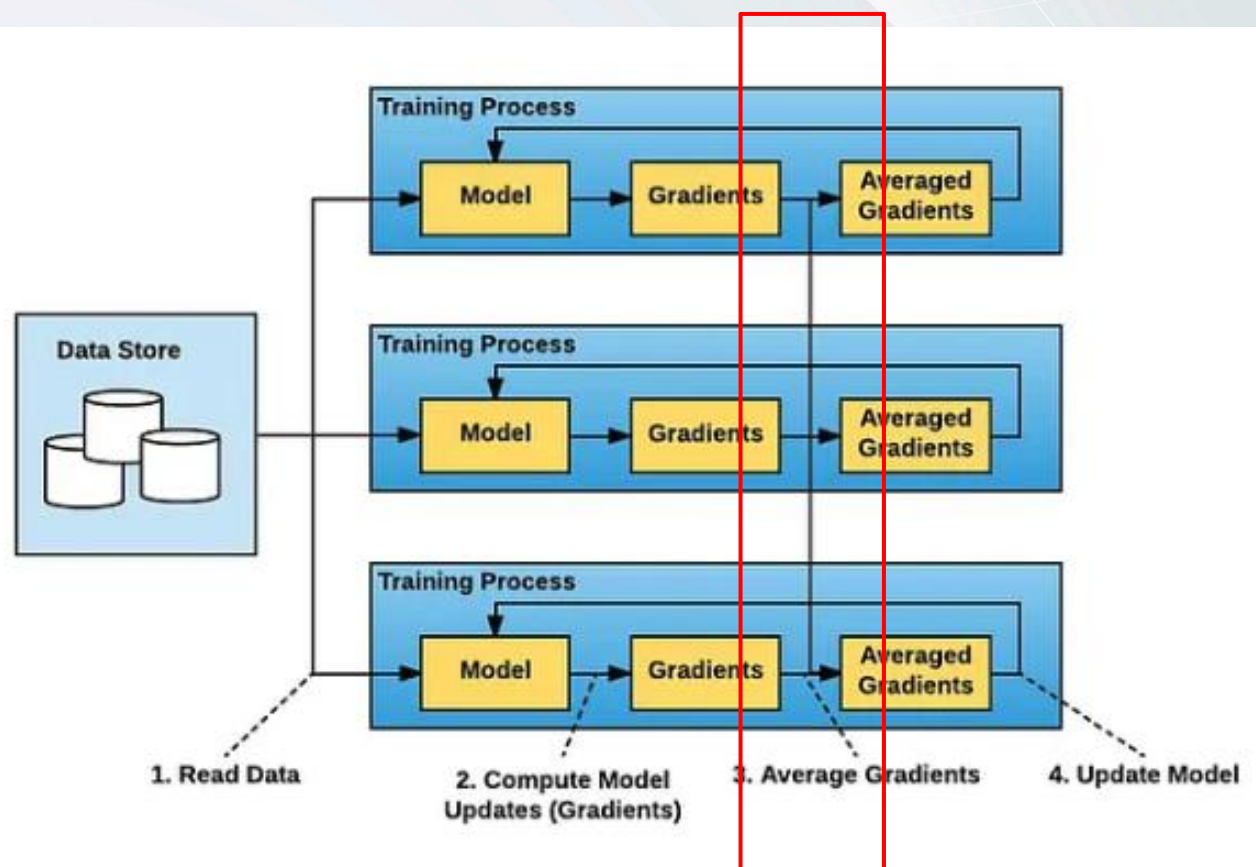
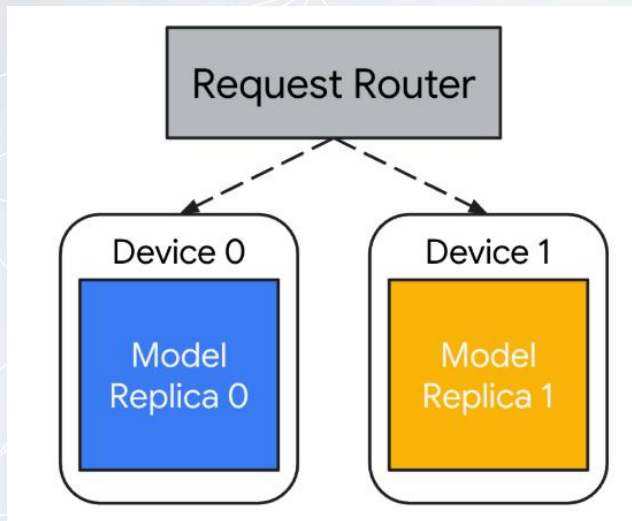


Take FSDP (PyTorch) as an example

- DP, DDP, FSDP: Data Parallelism, Distributed DP, Fully Sharded DP
- What is DP: multiple model copies on different devices.

DataParallel

```
class torch.nn.DataParallel(module, device_ids=None, output_device=None,  
dim=0)
```

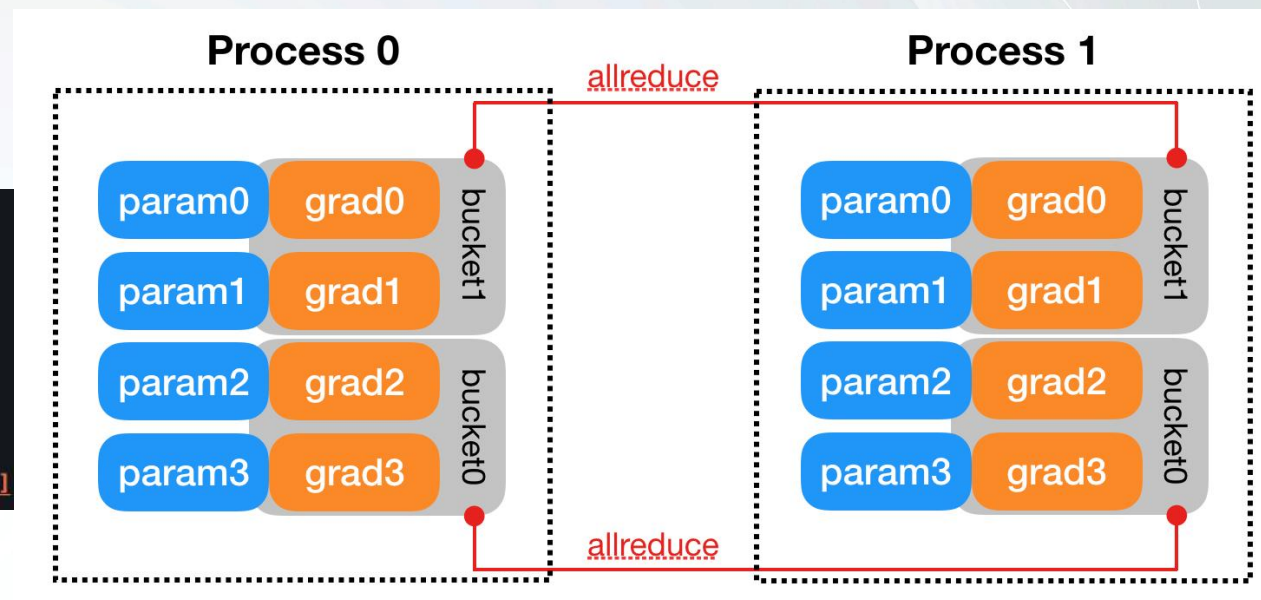


Take FSDP (PyTorch) as an example

- DP, DDP, FSDP: Data Parallelism, Distributed DP, Fully Sharded DP
- DDP is recommended: optimized all-reduce (ring-based) + distributed support
- DP: reduce op is executed on node 0 (imbalance); DDP: ring-based all-reduce.
- DDP: robust multi-process support.

DistributedDataParallel

```
class torch.nn.parallel.DistributedDataParallel(module, device_ids=None,
output_device=None, dim=0, broadcast_buffers=True, init_sync=True, process_group=None,
bucket_cap_mb=None, find_unused_parameters=False, check_reduction=False,
gradient_as_bucket_view=False, static_graph=False, delay_all_reduce_named_params=None,
param_to_hook_all_reduce=None, mixed_precision=None, device_mesh=None,
skip_all_reduce_unused_params=False)
```

[\[source\]](#)

Take FSDP (PyTorch) as an example

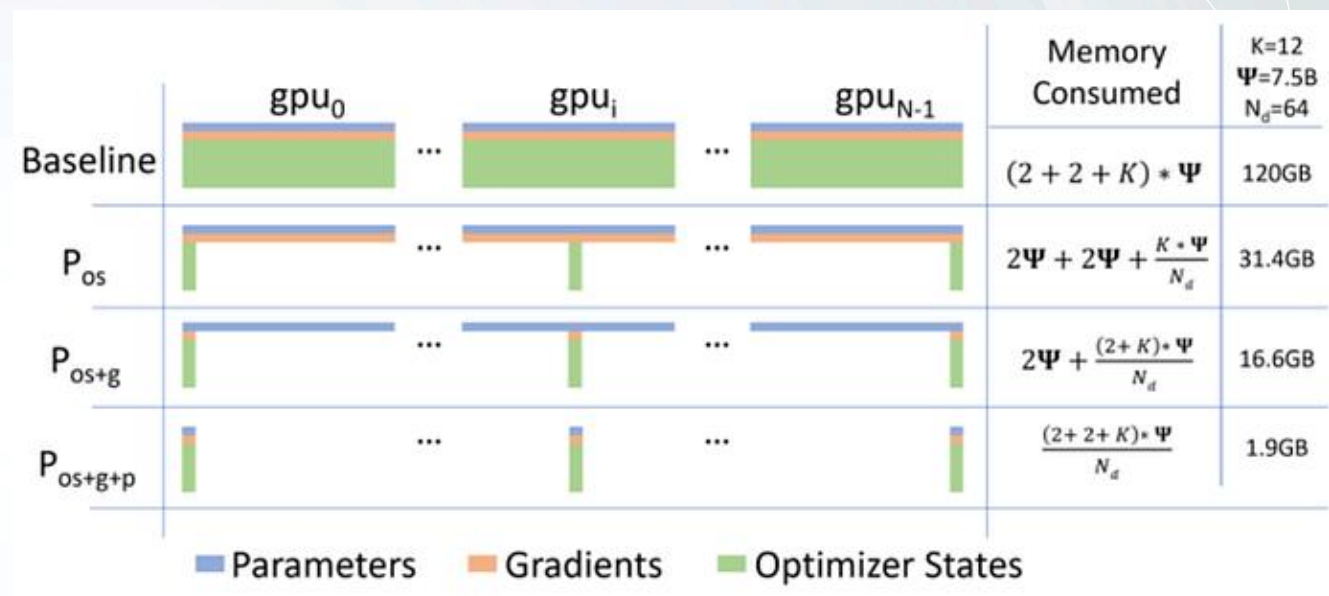
- DP, DDP, FSDP: Data Parallelism, Distributed DP, Fully Sharded DP
- Before FSDP, we profile the memory utilization during the LLM training.
 - Model params account for only a portion of the total memory consumption during training.
 - We need to consider the optimizer states and gradients.
- Core Idea: removing redundant data.
- ZeRO (Zero Redundancy Optimizer).



Take FSDP (PyTorch) as an example

- DP, DDP, FSDP: Data Parallelism, Distributed DP, Fully Sharded DP
- ZeRO with different opt levels:
 - ZeRO-1: Shards the optimizer states (Optimizer States Sharding).
 - ZeRO-2: Shards both optimizer states and gradients (Optimizer States & Gradients Sharding).
 - ZeRO-3: Shards the optimizer states, gradients, and model parameters (Optimizer States & Gradients & Parameters Sharding).

ZeRO3 will introduce the extra communications because of the partitions of model parameters.



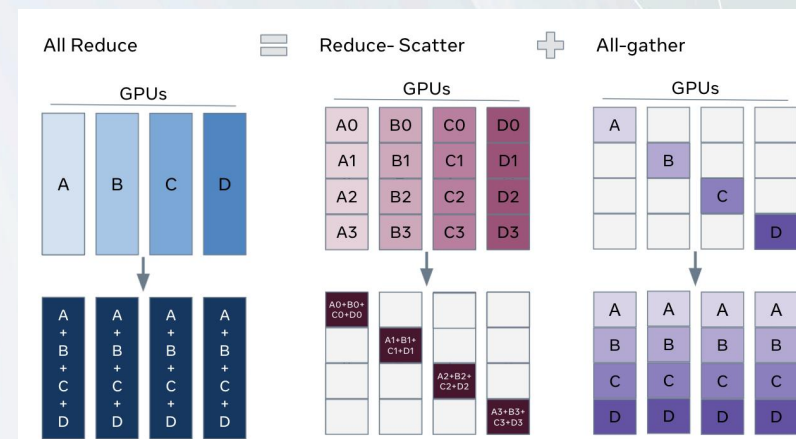
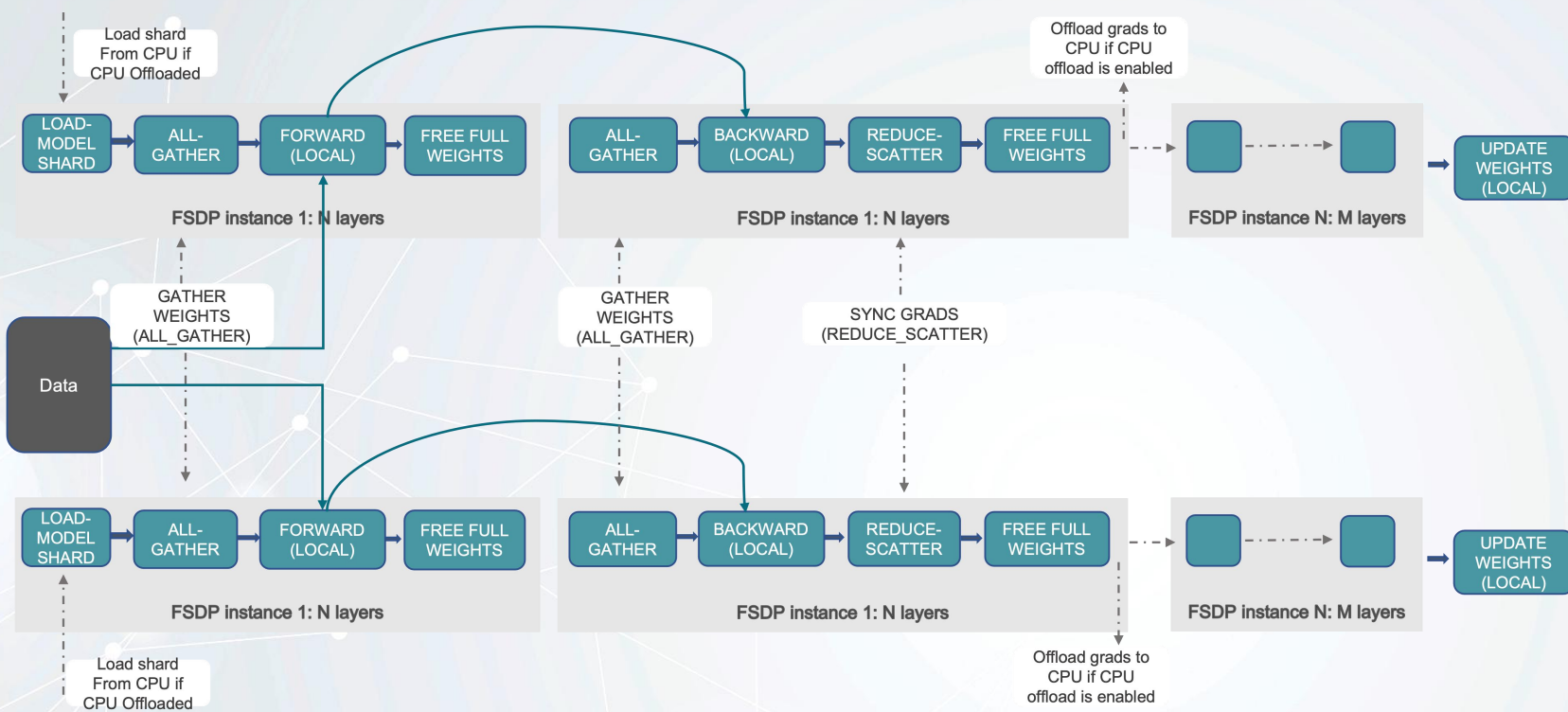
K denotes the memory multiplier of the optimizer states

[1] <https://docs.pytorch.org/docs/stable/notes/ddp.html>

[2] <https://www.youtube.com/watch?v=czgA-MbAdvA&t=2550s>

Take FSDP (PyTorch) as an example

- DP, DDP, FSDP: Data Parallelism, Distributed DP, Fully Sharded DP
- FSDP: sharding model parameters/gradient/optimizer states for GPU memory reduction (ZeRO3).

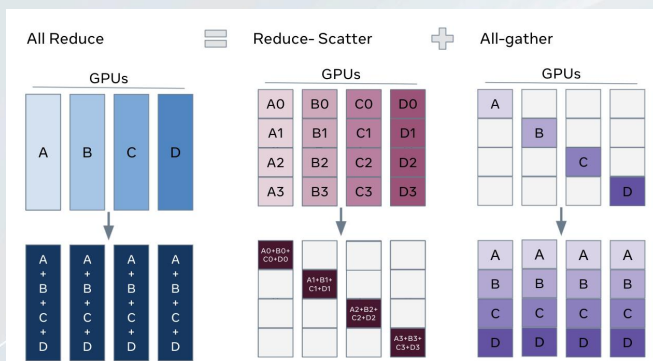


All-reduce =
Reduce-scatter + all-gather

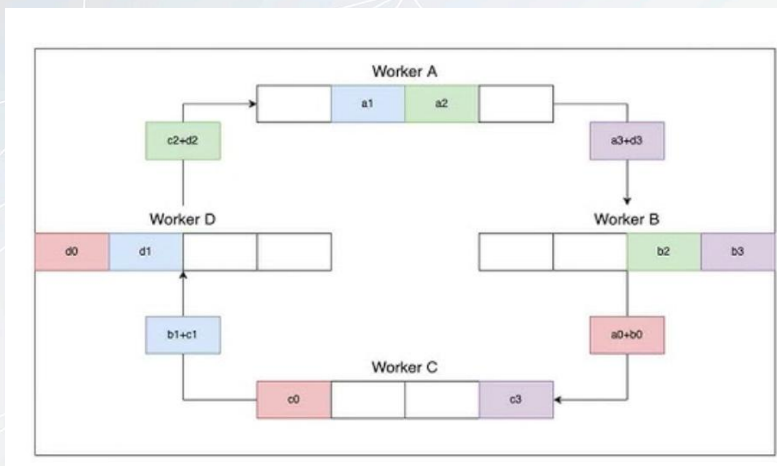
[1] https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html

[2] <https://arxiv.org/abs/2304.11277>

Implementations of collective communication ops

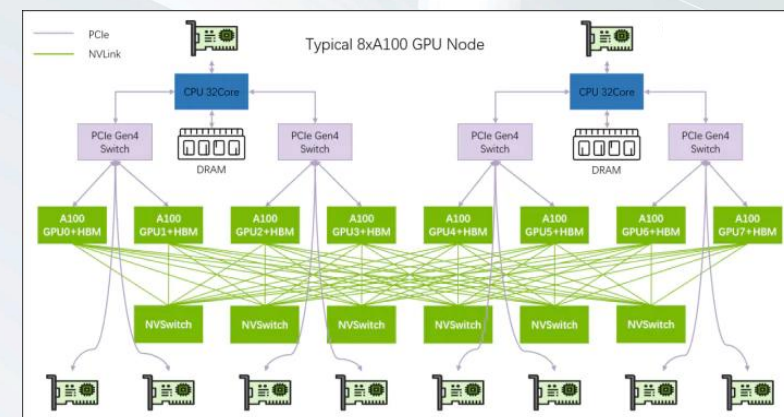


All-reduce = Reduce-scatter + all-gather



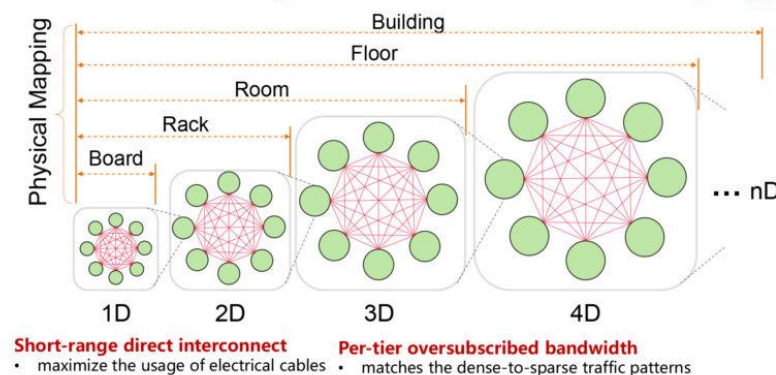
Ring All-reduce

The implementations highly depends on the hardware topology



NVIDIA GPU CLOS Topology

UB-Mesh: Hierarchically Localized nD-FullMesh Network Topology

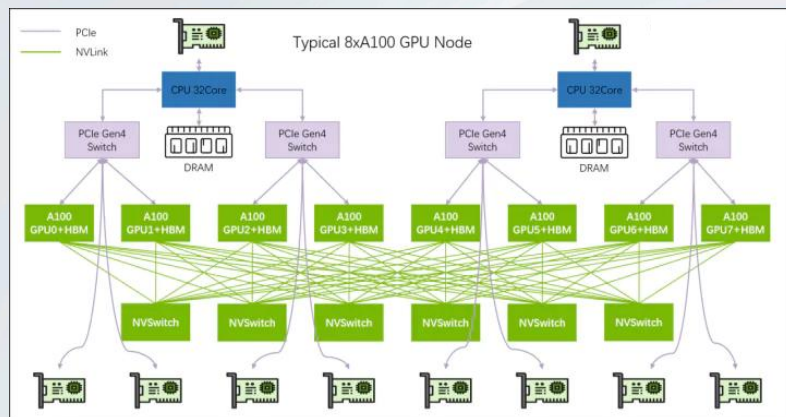


HUAWEI NPU Full-mesh Topology

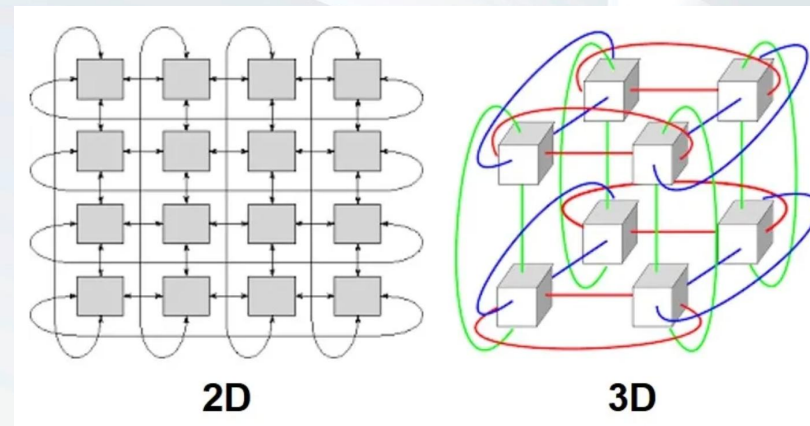
[1] https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html

[2] <https://arxiv.org/abs/2304.11277>

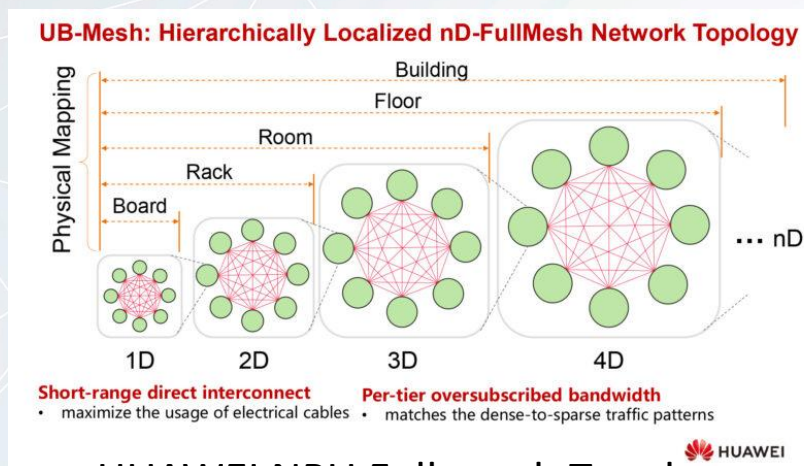
Topologies in the modern AI systems



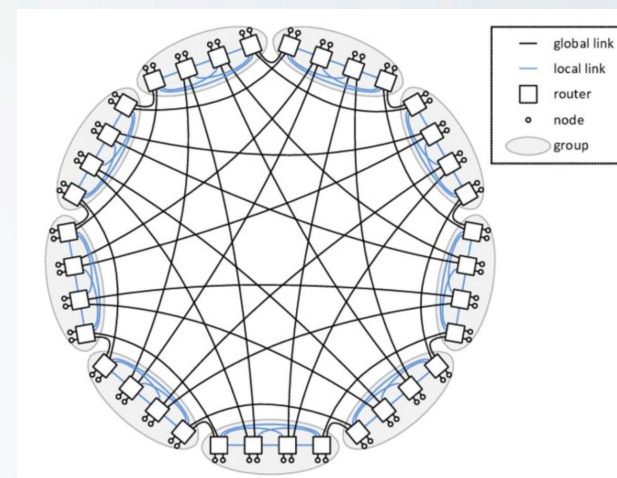
NVIDIA GPU Clos Topology



GOOGLE TPU Torus Topology



HUAWEI NPU Full-mesh Topology



Dragonfly Topology

Take AllReduce as an example

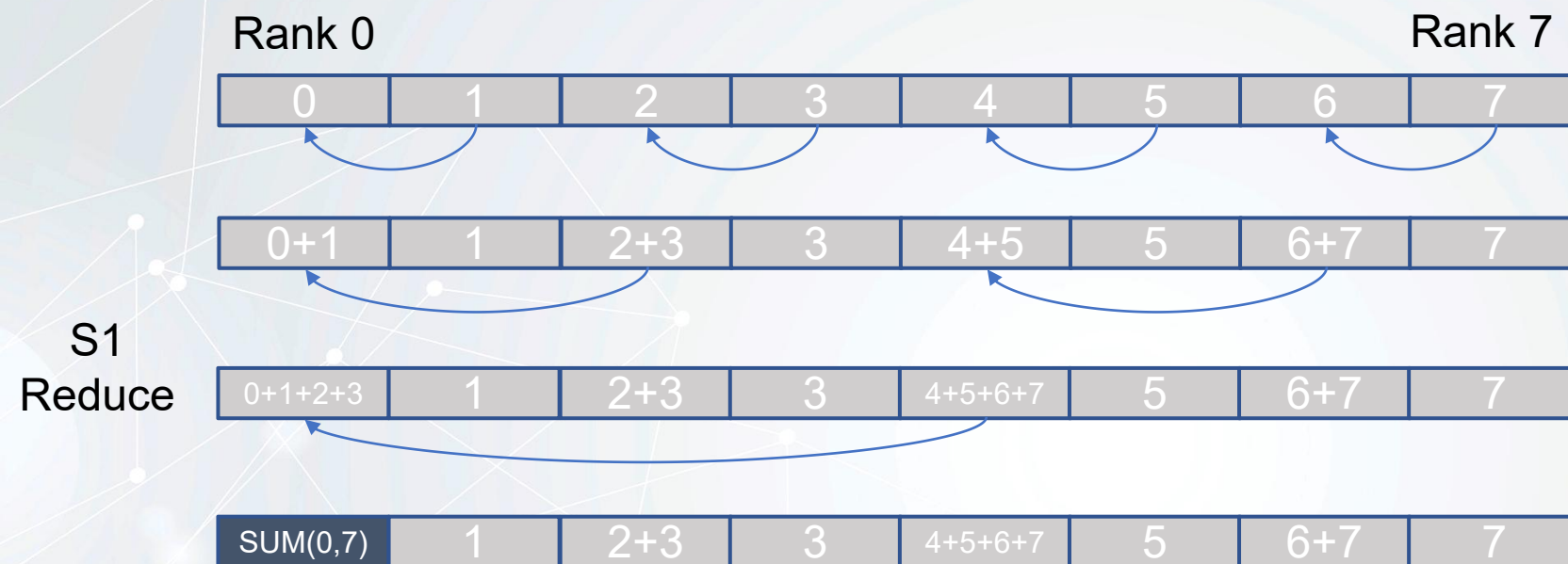


- Start from a naive implementation with root rank = 0.
 - Data transfer volume: $2 * (N-1) * K$
- There are three disadvantages
 - The bandwidth and computation of Rank 0 become bottleneck.
 - There are many idle resources during the allreduce (Rank 1 – N-1).
 - Poor scalability and lacks the consideration of topology.
- How can we improve it?
 - Scatter the data transfer and computation workload to all of the ranks.
- Solutions:
 - Tree-based, ring-based, halving-doubling, etc.

```
def naive_allreduce(local_data):  
    rank = get_rank()  
    num_procs = get_world_size()  
  
    if rank == 0:  
        global_sum = local_data  
        for i in range(1, num_procs):  
            received_data = recv_from(i)  
            global_sum += received_data  
    else:  
        send_to(rank=0, data=local_data)  
  
    if rank == 0:  
        for i in range(1, num_procs):  
            send_to(rank=i, data=global_sum)  
    else:  
        global_sum = recv_from(0)  
  
    return global_sum
```

Take AllReduce as an example

- Tree-based AllReduce Algorithm
 - Build a hierarchical relation with the root of tree (Rank 0).
 - Stage 1: Bottom-up Reduce
 - Stage 2: Top-down Broadcast



```
def tree_allreduce(my_data, rank, N):
    # --- Stage 1: Reduce (to root Rank 0) ---
    step = 1
    while step < N:
        if (rank & step) != 0: # Sender
            target = rank - step
            send(my_data, to=target)
            break
        else: # Receiver
            source = rank + step
            if source < N:
                received_data = rcv(from=source)
                my_data += received_data

        step *= 2

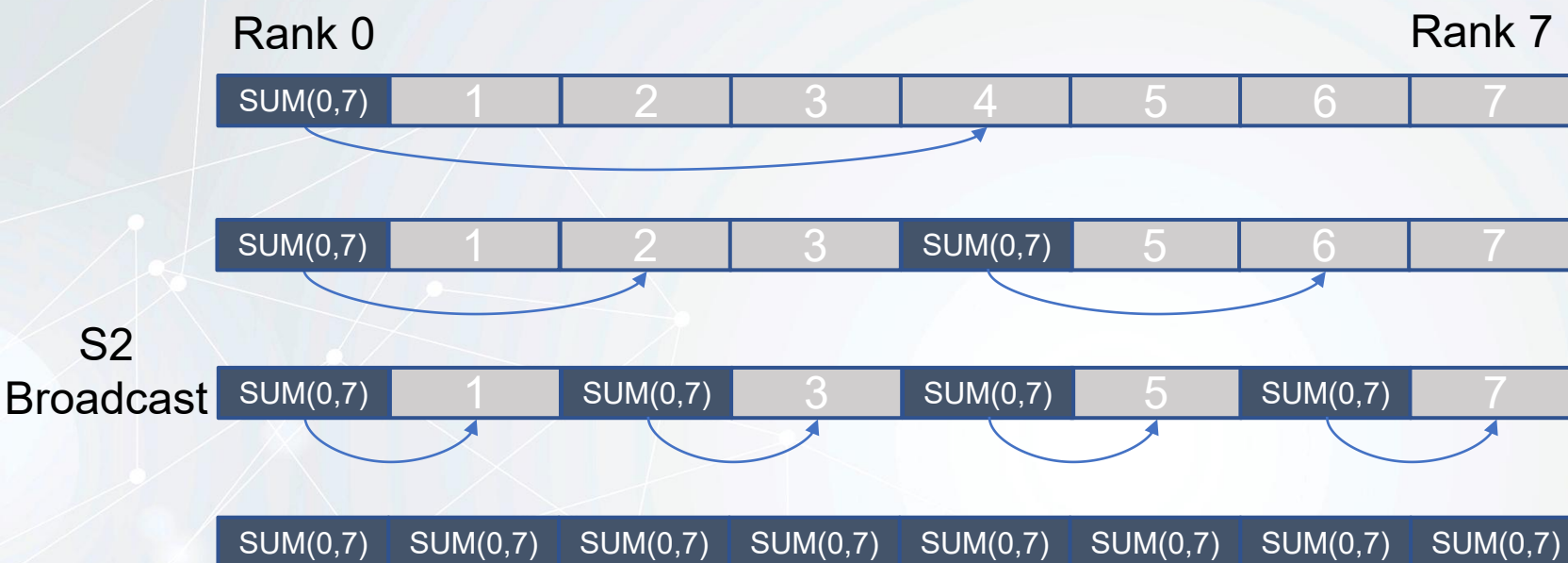
    # --- Stage 2: Broadcast (to all ranks) ---
    while step >= 1:
        if (rank & step) == 0: # Sender
            target = rank + step
            if target < N:
                send(my_data, to=target)
        else: # Receiver
            source = rank - step
            my_data = rcv(from=source)

        step //= 2

    return my_data
```

Take AllReduce as an example

- Tree-based AllReduce Algorithm
 - Build a hierarchical relation with the root of tree (Rank 0).
 - Stage 1: Bottom-up Reduce
 - Stage 2: Top-down Broadcast



```
def tree_allreduce(my_data, rank, N):
    # --- Stage 1: Reduce (to root Rank 0) ---
    step = 1
    while step < N:
        if (rank & step) != 0: # Sender
            target = rank - step
            send(my_data, to=target)
            break
        else: # Receiver
            source = rank + step
            if source < N:
                received_data = recv(from=source)
                my_data += received_data

        step *= 2

    # --- Stage 2: Broadcast (to all ranks) ---
    while step >= 1:
        if (rank & step) == 0: # Sender
            target = rank + step
            if target < N:
                send(my_data, to=target)
        else: # Receiver
            source = rank - step
            my_data = recv(from=source)

        step //= 2

    return my_data
```

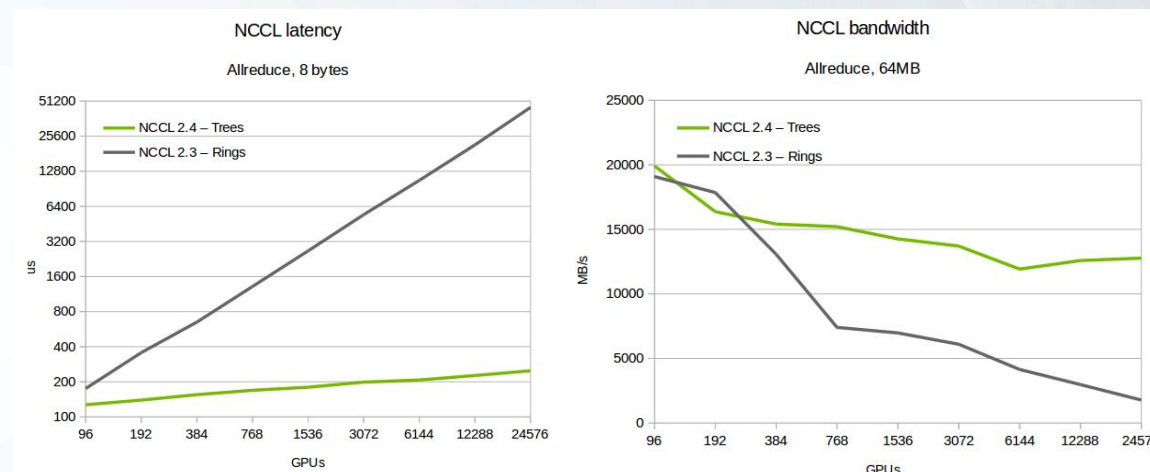
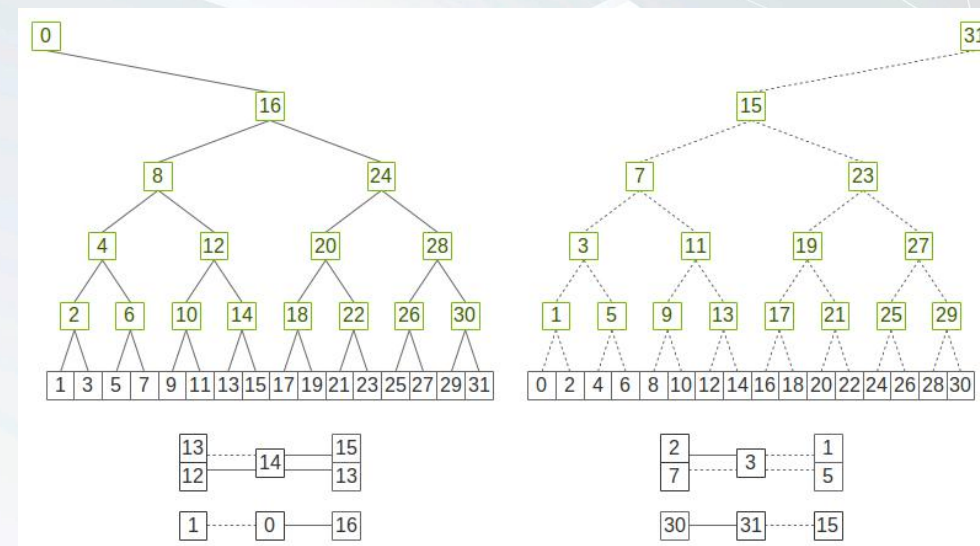
Take AllReduce as an example

- Tree-based AllReduce Algorithm
 - Build a hierarchical relation with the root of tree (Rank 0).
 - Stage 1: Bottom-up Reduce
 - Stage 2: Top-down Broadcast
- Advantages
 - Parallel data transfer and reduce operations.
 - Logarithmic complexity (the naive version has linear complexity).
 - It excels with small data sizes because of the low latency.
- Disadvantages
 - It still has idle computing and bandwidth resources.
- Further Opt: Double Binary Tree (NCCL)

```
def tree_allreduce(my_data, rank, N):  
    # --- Stage 1: Reduce (to root Rank 0) ---  
    step = 1  
    while step < N:  
        if (rank & step) != 0: # Sender  
            target = rank - step  
            send(my_data, to=target)  
            break  
        else: # Receiver  
            source = rank + step  
            if source < N:  
                received_data = recv(from=source)  
                my_data += received_data  
  
        step *= 2  
  
    # --- Stage 2: Broadcast (to all ranks) ---  
    while step >= 1:  
        if (rank & step) == 0: # Sender  
            target = rank + step  
            if target < N:  
                send(my_data, to=target)  
        else: # Receiver  
            source = rank - step  
            my_data = recv(from=source)  
  
        step //= 2  
  
    return my_data
```

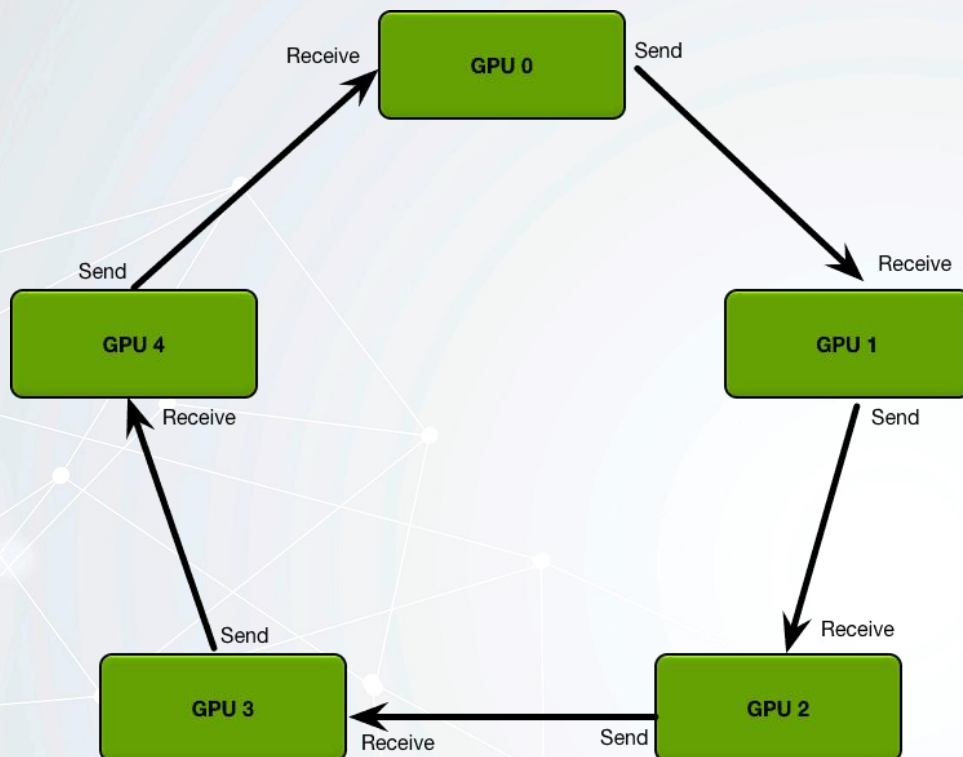
Double Binary Tree-based Allreduce

- Observation
 - in a binary tree: internal/leaves $\approx 50\%/50\%$
- Double binary tree
 - no rank is internal on both trees
 - all ranks have both two parents and two children except for the root
 - full bandwidth and logarithmic latency
 - good for small and medium size messages



Take AllReduce as an example

- Ring-based AllReduce Algorithm
 - Stage 1: Iterative Reduce
 - Stage 2: Iterative Gather

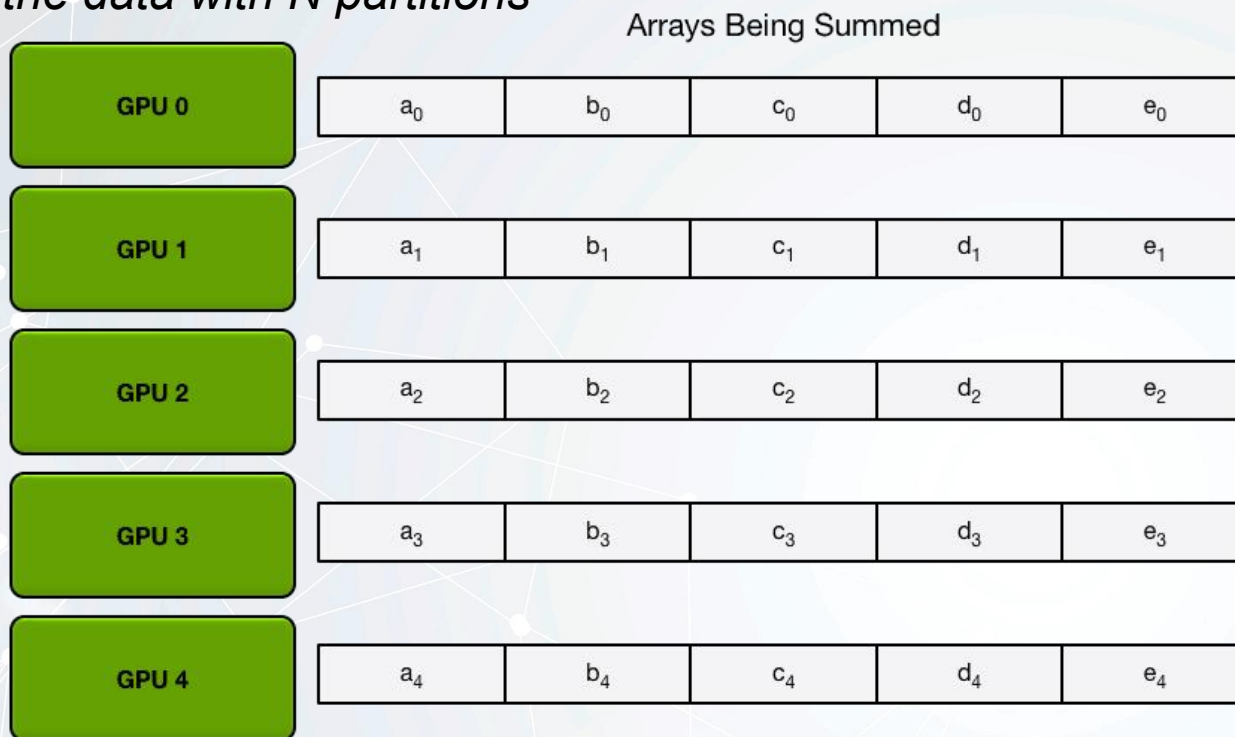


```
def ring_allreduce(local_data):  
    rank = get_rank()  
    N = get_world_size()  
  
    chunks = split_data(local_data, N)  
  
    send_to_rank = (rank + 1) % N  
    recv_from_rank = (rank - 1 + N) % N  
  
    # --- Stage 1: Scatter-Reduce ---  
    for i in range(N - 1):  
        send_idx = (rank - i + N) % N  
        recv_idx = (rank - i - 1 + N) % N  
  
        send_data(chunks[send_idx], dest=send_to_rank)  
        received_chunk = recv_data(src=recv_from_rank)  
  
        chunks[recv_idx] += received_chunk  
  
    # --- Stage 2: All-Gather ---  
    for i in range(N - 1):  
        send_idx = (rank - i + 1 + N) % N  
        recv_idx = (rank - i + N) % N  
  
        send_data(chunks[send_idx], dest=send_to_rank)  
        chunks[recv_idx] = recv_data(src=recv_from_rank)  
  
    return flatten(chunks)
```

Take AllReduce as an example

- Ring-based AllReduce Algorithm
 - Stage 1: Iterative Reduce with N-1 steps
 - Stage 2: Iterative Gather with N-1 steps

Split the data with N partitions



```
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

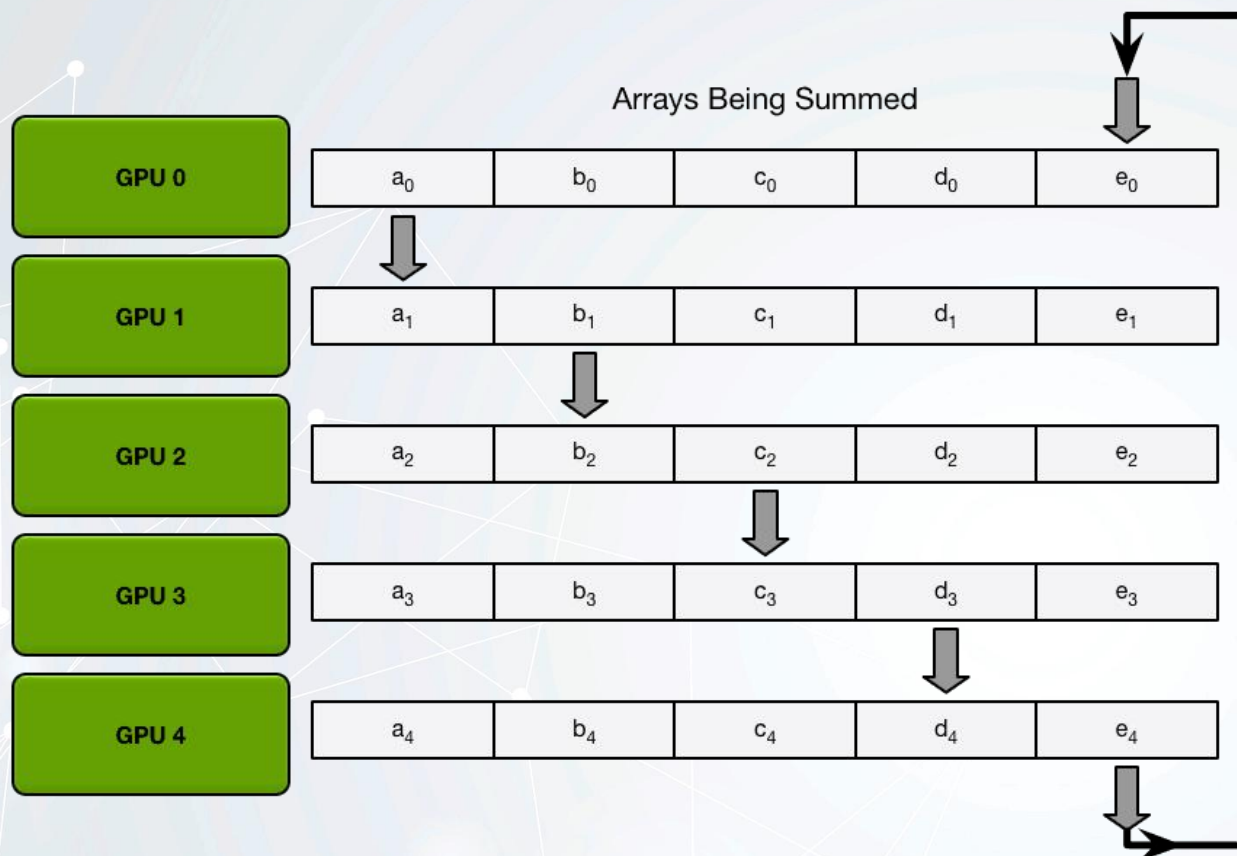
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

Take AllReduce as an example

- Ring-based AllReduce Algorithm
 - Stage 1: Iterative Reduce with N-1 steps
 - Stage 2: Iterative Gather with N-1 steps



```
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

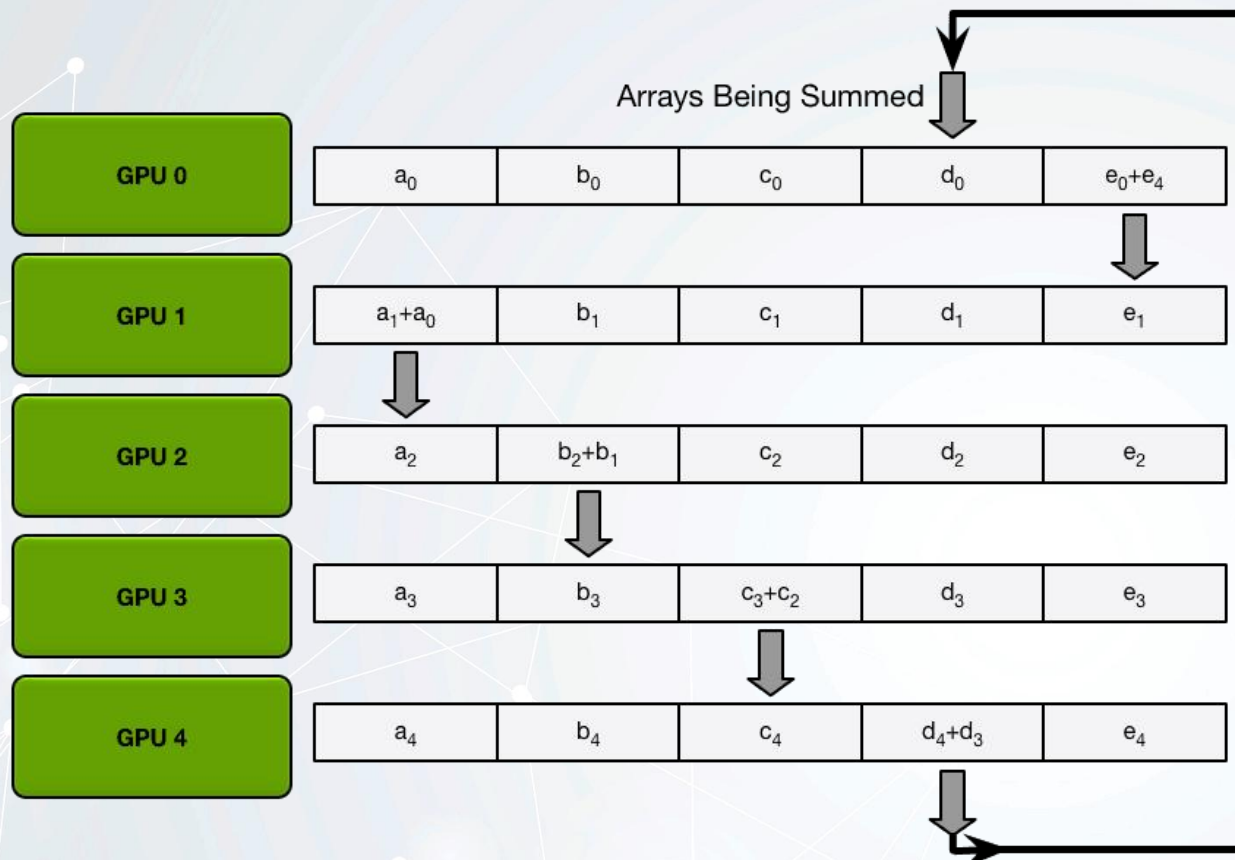
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

Take AllReduce as an example

- Ring-based AllReduce Algorithm
 - Stage 1: Iterative Reduce with $N-1$ steps
 - Stage 2: Iterative Gather with $N-1$ steps



```
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

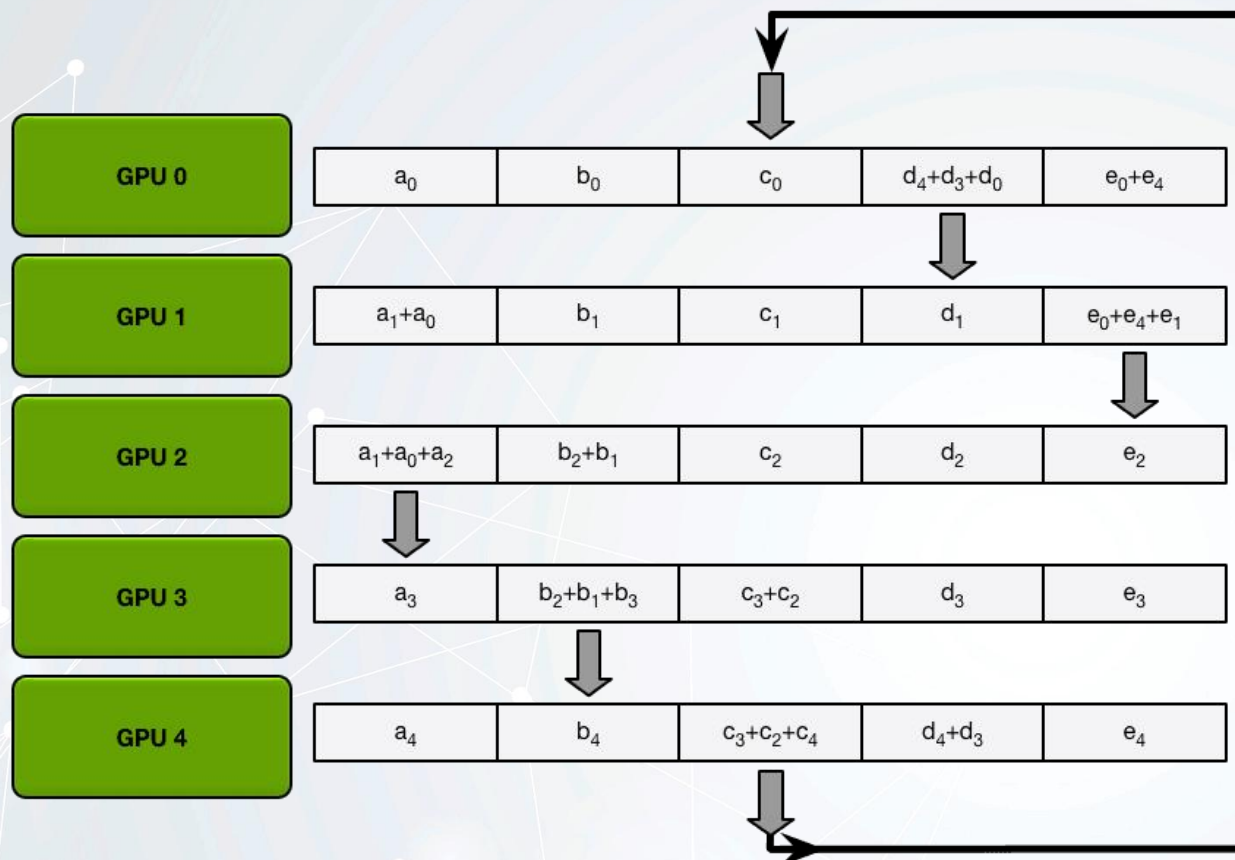
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

Take AllReduce as an example

- Ring-based AllReduce Algorithm
 - Stage 1: Iterative Reduce with $N-1$ steps
 - Stage 2: Iterative Gather with $N-1$ steps



```
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

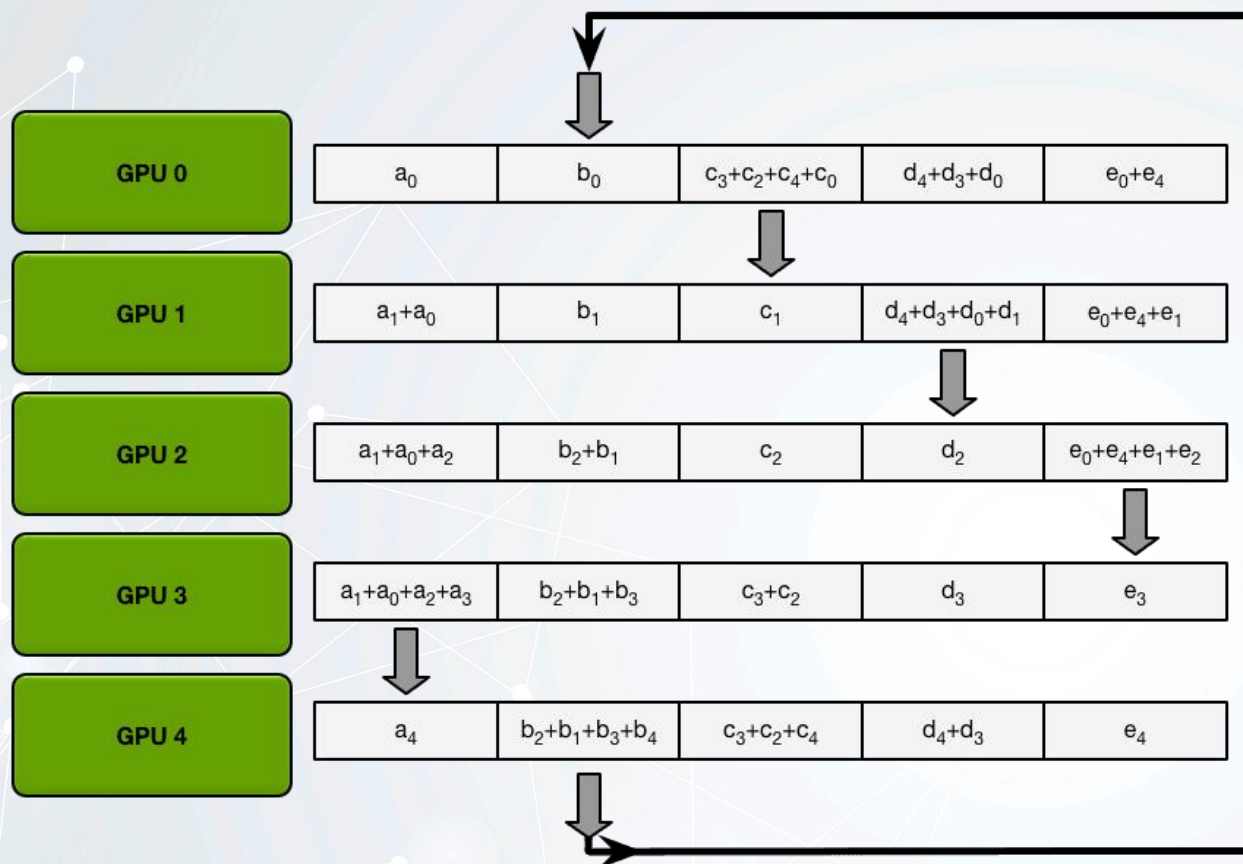
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

Take AllReduce as an example

- Ring-based AllReduce Algorithm
 - Stage 1: Iterative Reduce with N-1 steps
 - Stage 2: Iterative Gather with N-1 steps



```
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

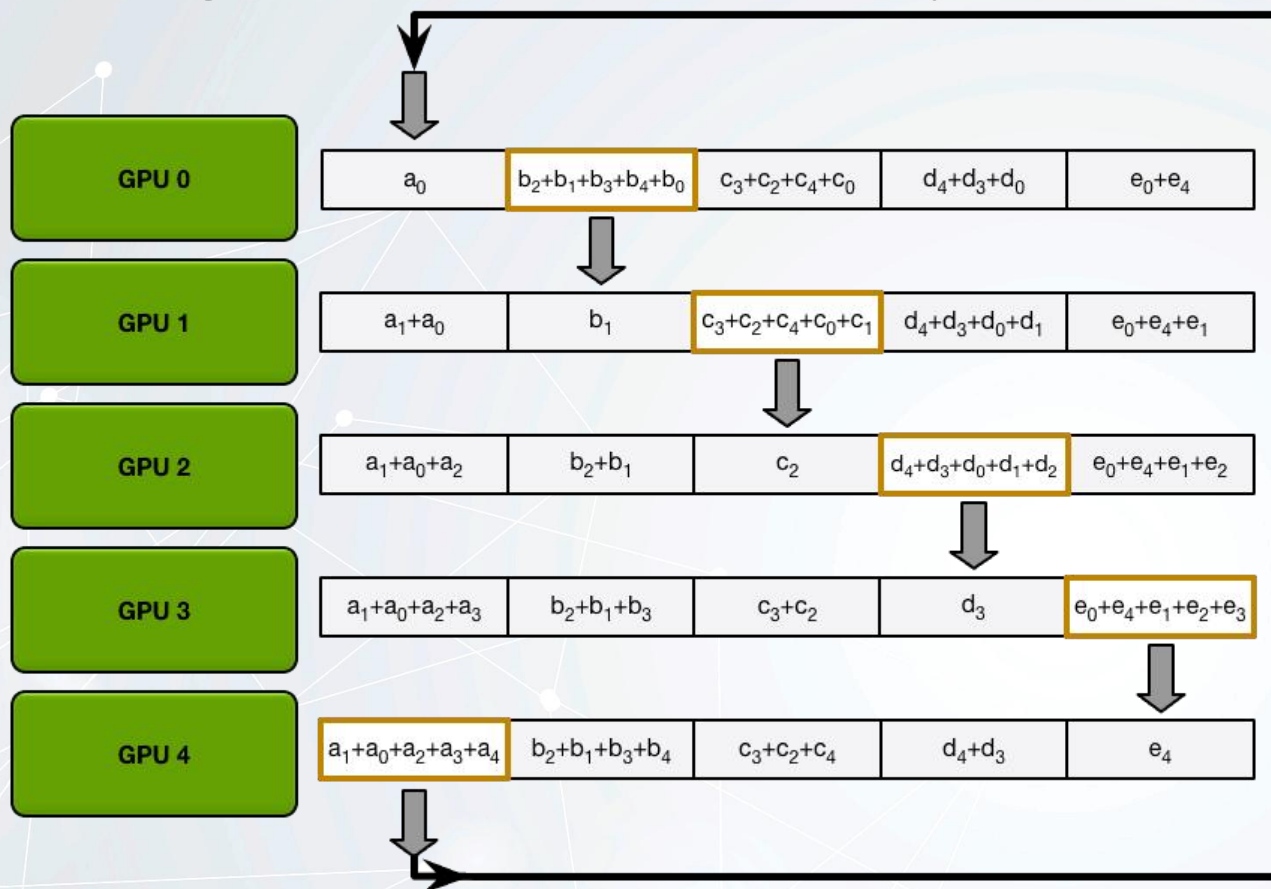
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

Take AllReduce as an example

- Ring-based AllReduce Algorithm
 - Stage 1: Iterative Reduce with N-1 steps
 - Stage 2: Iterative Gather with N-1 steps



```
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

Take AllReduce as an example

- Ring-based AllReduce Algorithm
 - Stage 1: Iterative Reduce with N-1 steps
 - Stage 2: Iterative Gather with N-1 steps
- Quantitative Analysis
 - Data Transfer Volume: $2 * (N-1) * K / N$
 - Overall Time: $2 * (N-1) * \alpha + 2 * (N-1) * K / N / \beta$
 - Tree-based AllReduce Time : $2 * \log(N) * (\alpha + K / \beta)$
 - α : latency; β : bandwidth
- Advantages: Optimal bandwidth utilization

```
def ring_allreduce(local_data):  
    rank = get_rank()  
    N = get_world_size()  
  
    chunks = split_data(local_data, N)  
  
    send_to_rank = (rank + 1) % N  
    recv_from_rank = (rank - 1 + N) % N  
  
    # --- Stage 1: Scatter-Reduce ---  
    for i in range(N - 1):  
        send_idx = (rank - i + N) % N  
        recv_idx = (rank - i - 1 + N) % N  
  
        send_data(chunks[send_idx], dest=send_to_rank)  
        received_chunk = recv_data(src=recv_from_rank)  
  
        chunks[recv_idx] += received_chunk  
  
    # --- Stage 2: All-Gather ---  
    for i in range(N - 1):  
        send_idx = (rank - i + 1 + N) % N  
        recv_idx = (rank - i + N) % N  
  
        send_data(chunks[send_idx], dest=send_to_rank)  
        chunks[recv_idx] = recv_data(src=recv_from_rank)  
  
    return flatten(chunks)
```

Take AllReduce as an example

- Ring-based AllReduce Algorithm
 - Stage 1: Iterative Reduce with N-1 steps
 - Stage 2: Iterative Gather with N-1 steps
- Quantitative Analysis
 - Data Transfer Volume: $2 * (N-1) * K / N$
 - Overall Time: $2 * (N-1) * \alpha + 2 * (N-1) * K / N / \beta$
 - Tree-based AllReduce Time : $2 * \log(N) * (\alpha + K / \beta)$
 - α : latency; β : bandwidth
- Advantages: Optimal bandwidth utilization

```
def ring_allreduce(local_data):  
    rank = get_rank()  
    N = get_world_size()  
  
    chunks = split_data(local_data, N)  
  
    send_to_rank = (rank + 1) % N  
    recv_from_rank = (rank - 1 + N) % N  
  
    # --- Stage 1: Scatter-Reduce ---  
    for i in range(N - 1):  
        send_idx = (rank - i + N) % N  
        recv_idx = (rank - i - 1 + N) % N  
  
        send_data(chunks[send_idx], dest=send_to_rank)  
        received_chunk = recv_data(src=recv_from_rank)  
  
        chunks[recv_idx] += received_chunk  
  
    # --- Stage 2: All-Gather ---  
    for i in range(N - 1):  
        send_idx = (rank - i + 1 + N) % N  
        recv_idx = (rank - i + N) % N  
  
        send_data(chunks[send_idx], dest=send_to_rank)  
        chunks[recv_idx] = recv_data(src=recv_from_rank)  
  
    return flatten(chunks)
```

Take AllReduce as an example

- Halving-doubling AllReduce Algorithm

- Stage 1: Recursive halving reduce
- Stage 2: Recursive doubling gather

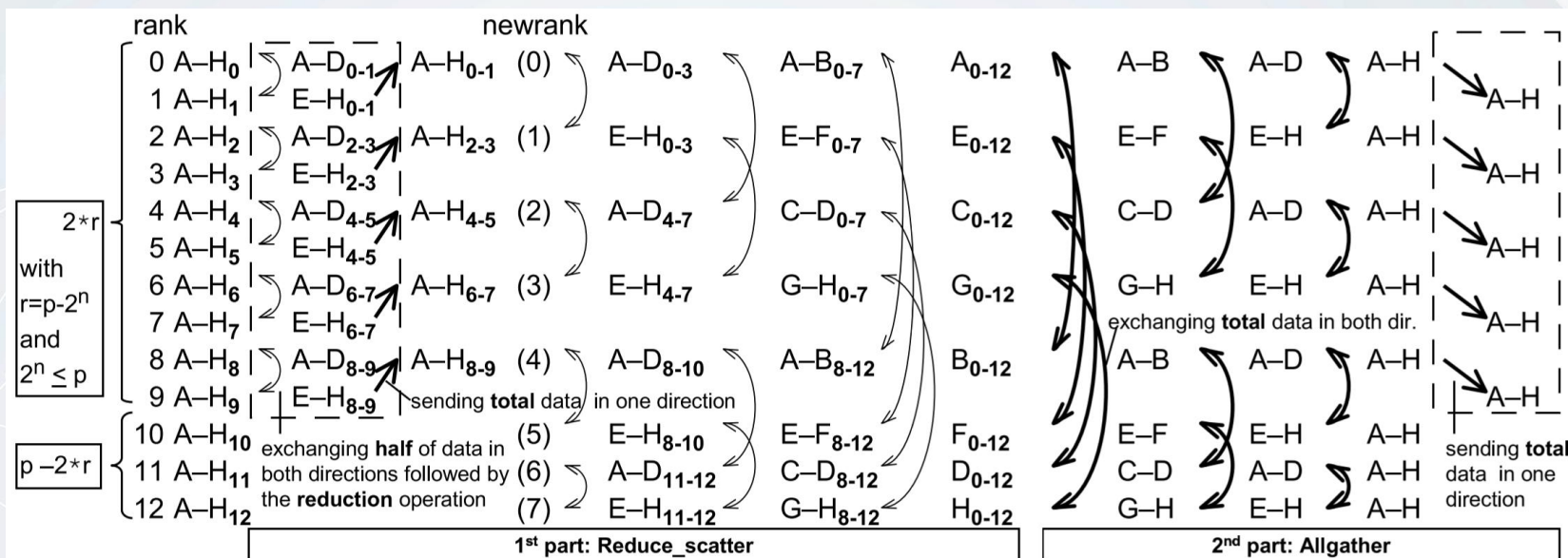


Figure 12: Allreduce using the recursive halving and doubling algorithm. The intermediate results after each communication step, including the reduction operation in the reduce-scatter phase, are shown. The dotted frames show the additional overhead caused by a non-power-of-two number of processes.

Take AllReduce as an example



- Comparisons among ring-based, tree-based, and halving-doubling Allreduce algorithms.
- Modern CCLs can select an efficient strategy according to the topology and the communication domain.

Algorithm	Steps (Complexity)	Latency	Bandwidth Efficiency	Ideal Data Size
Ring-based	$O(N)$	High	Optimal	Large Data (> 64MB)
Tree-based	$O(\log N)$	Lowest	Moderate	Small-Medium (< 16MB)
Halving-Doubling	$O(\log N)$	Low	Moderate	Very Small (< 1MB)

Protocol design matters

- Take NCCL as an example.

TABLE I
COMPARISON OF NCCL COMMUNICATION PROTOCOLS

	Simple	LL	LL128
Design Goal	High bandwidth	Low latency	Low latency and high bandwidth
Synchronization Mechanism	Memory fences (high overhead)	Flag-based synchronization	Flag-based synchronization
Payload	Data chunks	4B data + 4B flag	120B data + 8B flag
Bandwidth Utilization	Near peak	25 ~ 50% of peak [11]	~ 95% of peak [11]
Latency Per-hop	~ 6 μ s	~ 1 μ s	~ 2 μ s

TABLE II
NCCL COMMUNICATION CHARACTERISTICS AND TRANSPORTS

	Intra-Node		Inter-Node	
Transport	P2P	p2p.cc	NET	net_ib.cc
	SHM	shm.cc		net_socket.cc
	NVLS	nvls.cc	COLLNET	coll_net.cc
Physical Interconnect	NVLink PCIe		InfiniBand RoCE TCP/IP (Socket)	
Optimizations	GPUDirect P2P P2P_DIRECT		GPUDirect RDMA	

TABLE III
SUPPORTED ALGORITHMS AND PROTOCOLS FOR NCCL COLLECTIVE OPERATIONS

	AllReduce			Broadcast			Reduce			ReduceScatter			AllGather		
Algorithm	Simple	LL	LL128	Simple	LL	LL128	Simple	LL	LL128	Simple	LL	LL128	Simple	LL	LL128
Ring	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Tree	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
CollNet Direct	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
CollNet Chain	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
NVLS	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗
NVLS Tree	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

Legend: ✓ = Supported, ✗ = Not supported.

Besides, the protocol designs are also important

- Take NCCL as an example.

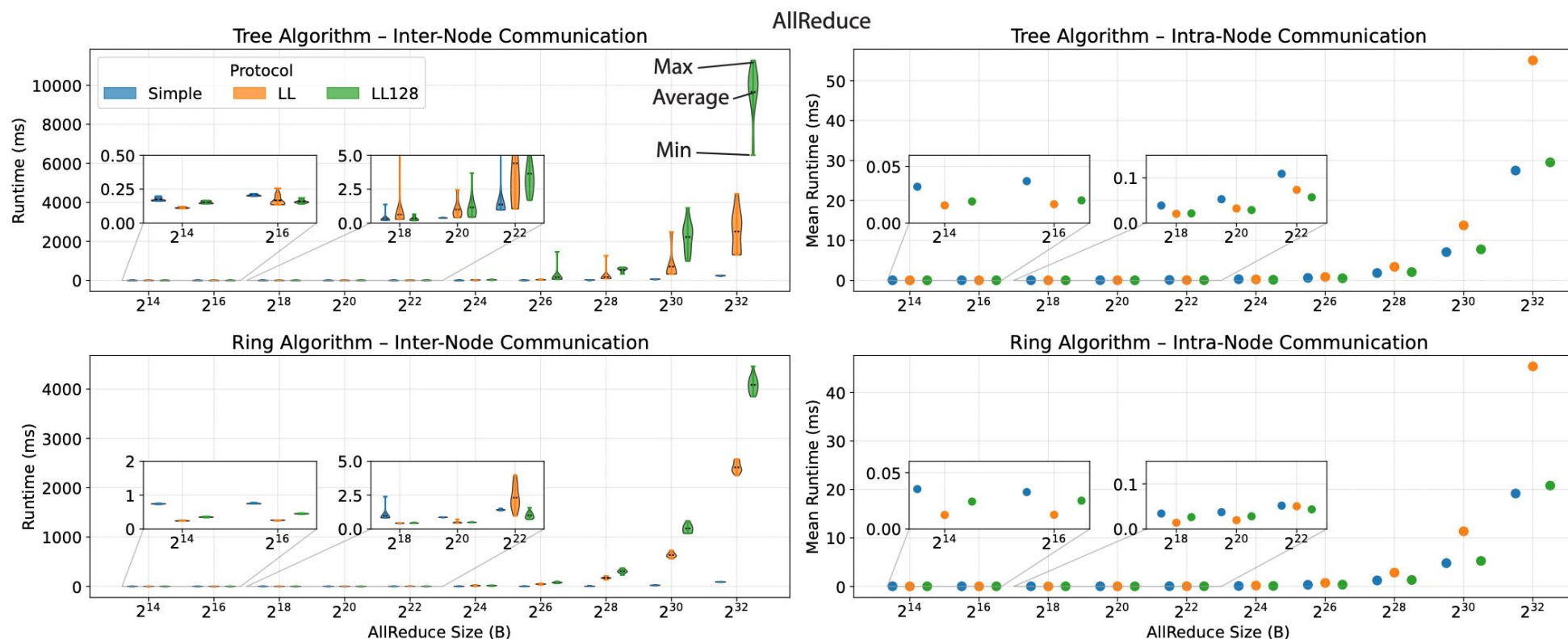


Fig. 6. Runtime comparison of protocols for Ring and Tree AllReduce when running inter- and intra-node. Each data point consists of 20 runs with a warm-up phase. For intra-node communication we report only the median value for readability as the variance is very low.

Broadcast (a.k.a., One-to-All Boardcast)

- Among a group of processing elements (nodes), a designated root node has a data item to be communicated (copied) to all other nodes.

Before			After		
Node r	Node 1	Node 2	Node r	Node 1	Node 2
x			x	x	x

- All nodes are assumed to explicitly take part in the broadcast operation.
- It is generally assumed that before its execution all nodes know the index of the designated root node as well as the the amount n of data to be broadcast.
- The data item x may be either a single, atomic unit or divisible into smaller, disjoint pieces.
 - The latter can be exploited algorithmically when n is large.

Broadcast: Assumptions

- Assumptions in the following analysis (for this lecture ONLY)
 - All nodes can communicate through a communication network.
 - Individual nodes perform communication operations that send and/or receive individual messages.
 - Communication is through a single port
(such that a node can be involved in at most one communication operation at a time)
 - Such an operation can be either a send to or a receive from another node (unidirectional communication),
 - a combined send to and receive from another node (bidirectional, telephone like communication), or
 - a send to and receive from two possibly different nodes (simultaneous sendreceive, fully bidirectional communication).
 - The communication medium is homogeneous and fully connected
 - A first approximation for the transferring time between two nodes: $\alpha + n\beta$
 - n is the message size
 - α is the start-up cost (latency)
 - β is the cost per item transferred (inverse of the bandwidth)

Broadcast: Two Lower Bounds

- Two lower bounds

- for the α term: $\lceil \log_2 p \rceil \alpha$

- a round: during which each node can send at most one message and receive at most one message.
 - In each round, the number of nodes that know message x can at most double.
 - Thus, a minimum of $\lceil \log_2 p \rceil$ rounds are needed to broadcast the message.
 - Each round costs at least α .

- for the β term: $n\beta$

- If $p > 1$ then the message must leave the root node, requiring a time of at least $n\beta$.

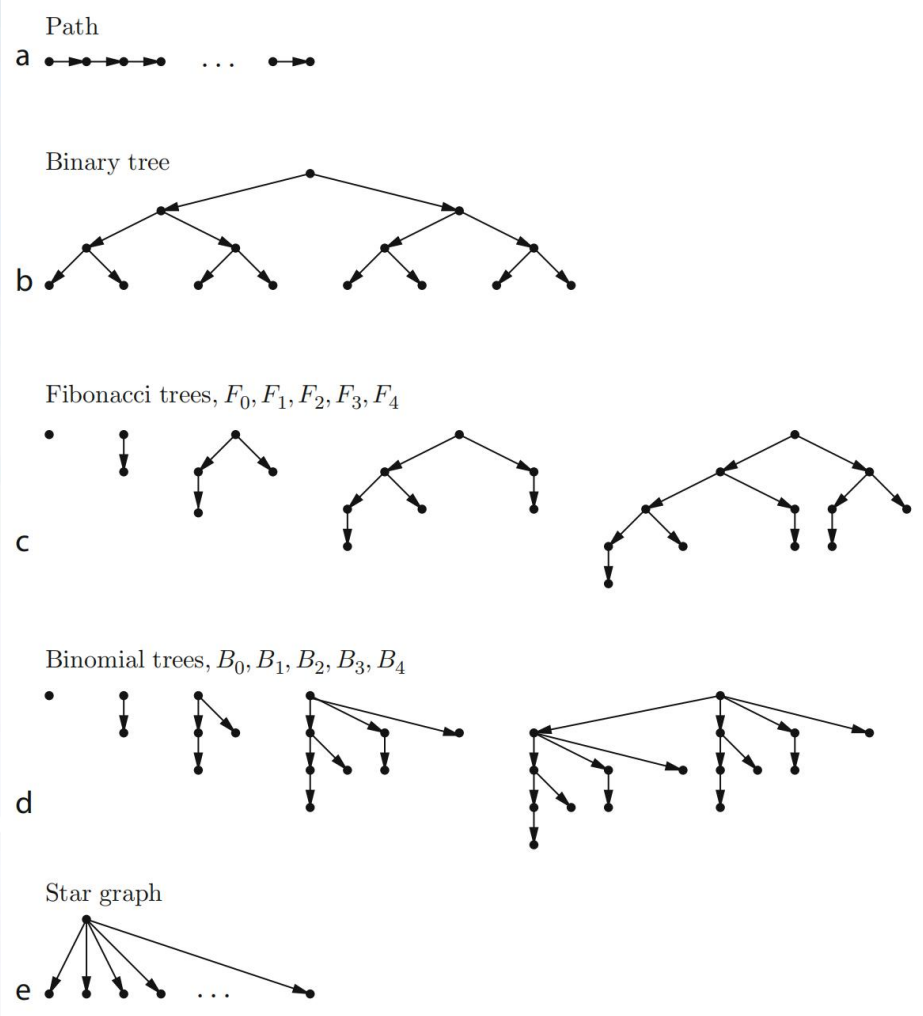
- Notes

- When assumptions about the communication system change, these lower bounds change as well.
 - Lower bounds for mesh and torus networks, hypercubes, and many other topologies are known^[WGe95].
 - Determining the minimum broadcast time in an arbitrary, given graph is NP-hard^[JaM95]

[WGe95] Watts J, van de Geijn RA (1995) A pipelined broadcast for multi-dimensional meshes. Parallel Process Lett 5: 281–292

[JaM95] Jansen K, Müller H (1995) The minimum broadcast time problem for several processor networks. Theor Comput Sci 147(1&2): 69–85

Broadcast: Tree-based Algorithms



Broadcast: The “MST” Algorithm

- The so-called* Minimum Spanning Tree (MST) algorithm
 - Partition the set of nodes into two roughly equalized subsets.
 - Send x from the root to a node in the subset that does not include the root.
 - The receiving node will become a local root in that subset.
 - Recursively broadcast x from the (local) root nodes in the two subsets.
- The total cost of the MST algorithm: $\lceil \log_2 p \rceil (\alpha + n\beta)$
 - It achieves the lower bound for the α term
 - but not for the β term for which it is a logarithmic factor off from the lower bound.
- The MST algorithm constructs a binomial spanning tree over the set of nodes
 - Figure (d) in the previous slide

* which is a misnomer, since all broadcast trees are spanning trees and minimum for homogeneous communication media

Broadcast: Pipelining



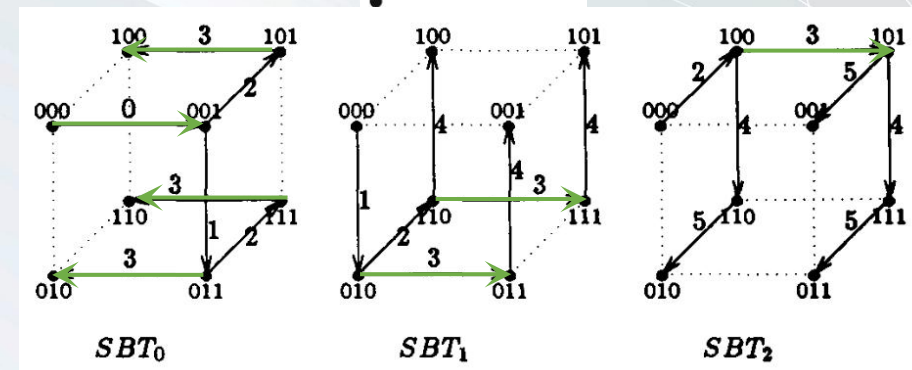
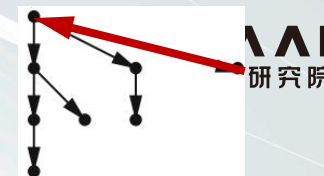
- For large n , pipelining improves the broadcast cost
 - the message is split into k blocks of size n/k each
- In the “path” broadcast tree:
 - The first piece from node 0 takes $p-1$ rounds to reach node $p-1$
 - an additional $k-1$ rounds for the remaining $k-1$ pieces
 - total time cost = $(k + p - 2)(\alpha + \beta n/k) \geq (p - 2)\alpha + 2\sqrt{(p - 2)n\alpha\beta} + \beta n$
 - with optimal number of blocks $k = \sqrt{(p - 2)\beta n/\alpha}$
- Pipelining can be applied on fixed degree trees
 - Binary tree: best-effort time cost = $2(\log_2 p - 1)\alpha + 4\sqrt{(\log_2 p - 1)n\alpha\beta} + 2\beta n$
 - Fibonacci tree: best-effort time cost = $(\log_\theta p - 1)\alpha + 2\sqrt{2(\log_\theta p - 1)n\alpha\beta} + 2\beta n$ where $\theta = \frac{\sqrt{5}+1}{2}$
- Pipelining is **not** helpful on nonconstant degrees trees (e.g., binomial tree and star graph)

Broadcast: the Third Lower Bound

- The third lower bound on the number of rounds: $k - 1 + \lceil \log_2 p \rceil$
 - first piece $\lceil \log_2 p \rceil$ rounds plus $k-1$ additional rounds
 - total time cost = $(k - 1 + \lceil \log_2 p \rceil)(\alpha + \beta n/k) \geq (\log_2 p - 1)\alpha + 2\sqrt{(\log_2 p - 1)\alpha}\sqrt{\beta n} + \beta n$
 - with optimal number of blocks $k = \sqrt{(\log_2 p - 1)\beta n/\alpha}$

Broadcast: Simultaneous Trees

- optimal $k - 1 + \log_2 p$ rounds of pipelining
 - when the message is split into k blocks
- $p=2^d$ nodes in a d -dimensional hypercube
 - node id: $i \in \{0, 1, \dots, 2^d-1\}$ with i_j as the j -th bit
- d edge-disjoint spanning binomial trees (ESBT)
 - all trees are rooted at node 0
 - the j -th tree has $2^{d-1}-1$ leaves with $i_j=0$
 - the j -th tree broadcasts blocks $j, j+d, j+2d, \dots$
- Leaves in tree $j = t \% d$ receive block $t-d$ in round t
 - block $-d, 1-d, \dots, -1, 0, 1, \dots, k-1$
- $\text{BitDistance}_i[j]$: distance to the next 1 to the left of i_j
 - $\text{BitDistance}_0[k] \equiv k$ for $0 < k \leq d$



```

f ← ((k mod d) + d - 1) mod d /* Start round for first phase */
t ← 0
while t < k + d - 1 do
  /* New phase consisting of (up to) d rounds */
  for j ← f to d - 1
    s ← t - d + (1 - i_j) * BitDistance_i[j] /* block to send */
    r ← t - d + i_j * BitDistance_i[j] /* block to receive */
    if s ≥ k then s ← k - 1
    if r ≥ k then r ← k - 1
    par /* simultaneous send-receive with neighbor */
      if s ≥ 0 then Send block s to node (i xor 2^j)
      if r ≥ 0 then Receive block r from node (i xor 2^j)
    end par
    t ← t + 1 /* next round */
  end for
  f ← 0 /* next phases start from 0 */
endwhile

```

Further Readings

- van de Geijn, R., Träff, J. (2011). Collective Communication. In: Padua, D. (eds) Encyclopedia of Parallel Computing. Springer, Boston, MA.
- Träff, J.L., van de Geijn, R.A. (2011). Broadcast. In: Padua, D. (eds) Encyclopedia of Parallel Computing. Springer, Boston, MA.
- Träffe, J.L., van de Geijn, R.A. (2011). Allgather. In: Padua, D. (eds) Encyclopedia of Parallel Computing. Springer, Boston, MA.
- Träff, J.L., vande Geijn, R.A. (2011). All-to-All. In: Padua, D. (eds) Encyclopedia of Parallel Computing. Springer, Boston, MA.